BugPoCer Scan Report

Prepared for: **CMTA**

Prepared by: **Olympix**

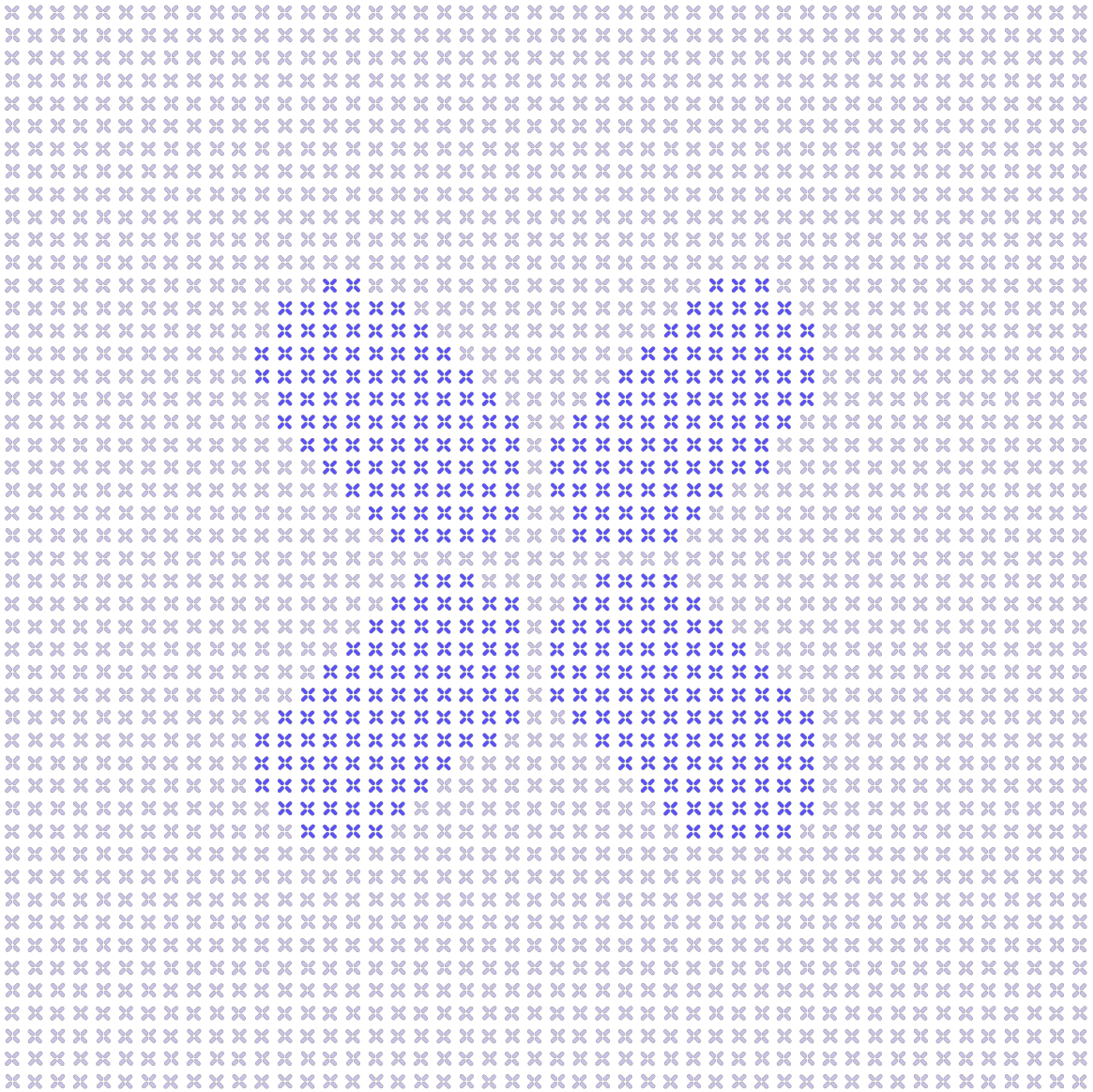


Table of Contents

1. About

This report was generated by Olympix's BugPoCer tool. BugPoCer automates end-to-end vulnerability exploitation for smart contracts - moving beyond static warning to deliver concrete, verifiable Proof-of-Concept (PoC) exploits. Built on Olympix's proprietary infrastructure and Intermediate Representation (IR), BugPoCer combines results from multiple analysis engines to identify confirmed security issues. Using an agentic AI framework to synthesize insights across the Olympix toolchain, it enables teams to instantly validate vulnerabilities, measure real-world impact, and confirm that patches are effective.

1.1. Methodology

BugPoCer breaks down a codebase into source *units* (function, contract, or library level). While findings will only be searched for in units within the specified scope, BugPoCer explores all relevant paths within the codebase. It couples this context with an extensive knowledge store to identify potential issues. For each issue, BugPoCer automatically generates a ready-to-run Foundry-format PoC test that reproduces the exploit under realistic attack conditions.

Each finding in the Scan Report includes:

- **Vulnerability Details** — Description, severity, affected lines, and potential impact
- **Exploit demonstration** — A PoC test that reliably triggers the issue

The findings within this report are broken down into 3 categories:

1. **True Positive (TP):** Vulnerabilities that were proven true via failing PoC test
2. **Needs Further Review:** Vulnerabilities for which BugPoCer was unable to write a compiling PoC test
3. **False Positive (FP):** Vulnerabilities that were proven false via passing PoC test

Findings marked true positive should be taken into careful consideration. Findings that need further review should be reviewed as potential exploits. BugPoCer, as any human auditor, can make mistakes. Therefore, false positives may be allocated some review time as a sanity check.

1.2. Disclaimer

This report does not guarantee the discovery of every vulnerability in scope, nor does it ensure that no new issues will arise as the codebase evolves. BugPoCer cannot provide assurances about code added or modified in future iterations. We recommend ongoing security reviews and the use of additional security measures such as independent audits and bug bounty programs.

Findings and suggested remediations in this report are provided for guidance only. Code samples or PoCs are illustrative and may require further testing and adaptation before production use.

This report is intended solely for informational purposes. It should not be interpreted as legal, financial, or investment advice, nor as an endorsement of the audited project.

1.3. Project Overview

Type: Security Token / Real-World-Asset Tokenization Framework (compliant ERC-20 security token)

Description: CMTAT is a modular security-token framework for issuing and managing tokenized regulated financial instruments (equities, debt/bonds, structured products, stablecoins, funds) on EVM chains, adding compliance controls (pause, account freeze, partial freeze, conditional/rule-based transfers, forced transfer/burn, documents, snapshots) on top of ERC-20 and standards such as ERC-3643 (without on-chain identity), ERC-1404, ERC-7551, ERC-7943, ERC-1363, ERC-7802 and ERC-2771.

Design Goals:

- Suitable for various regulated financial instruments (equities, debt, structured products, stablecoins) and adaptable to any jurisdiction
- Easy to customize and adapt for specific use cases through a modular architecture (modules + engines)
- Interoperability with the Ethereum ecosystem by implementing recognized standards (ERC-20, ERC-3643, ERC-7943, ERC-1404, ERC-7551, ERC-1363, ERC-2771, ERC-7201, ERC-7802)
- Security through third-party audits (ABDK, Halborn), static analysis, and industry best practices, with RBAC separation of roles
- Freedom of use via an open-source weak copyleft license (MPL-2.0)
- Keep contract size within the 24 kB EVM limit, driving decisions to externalize logic into engines and omit some functions

Key Patterns:

- ERC20 (Upgradeable)
- ERC-3643 (T-REX Permissioned Security Token)
- ERC-1404 (Simple Restricted Token) + RuleEngine compliance
- ERC-1363 (Payable Token / transferAndCall)
- Cross-Chain: ERC-7802 + Chainlink CCIP token admin

External Dependencies:

- Chainlink CCIP (Cross-Chain Interoperability Protocol / Token Bridge)
- RuleEngine (external ERC-3643/ERC-1404 compliance contract)
- SnapshotEngine (external balance-checkpointing contract)
- DocumentEngine (external ERC-1643 document registry)
- DebtEngine (external debt / credit-events data source)
- ERC-2771 Trusted Forwarder (meta-transaction relayer)

Invariants

High — funds/access risk Medium — functional correctness Low — informational

AI-Inferred Invariants *Properties inferred by automated analysis of code patterns, architecture, and documentation.*

> `totalSupply()` MUST always equal the sum of all individual `balanceOf` values after every `mint`, `burn`, `transfer`, `forcedTransfer`, `crosschainMint`, and `crosschainBurn` operation.

> Only an account holding `MINTER_ROLE` may increase supply via `mint` / `mintBatch`, and only an account holding `BURNER_ROLE` may reduce supply via `burn` / `burnBatch`.

> `crosschainMint` and `crosschainBurn` (ERC-7802) MUST only be callable by the authorized token bridge holding `CROSS_CHAIN_ROLE` (enforced by `onlyTokenBridge`), and the amount minted on the destination chain MUST equal the amount burned on the source chain.

> Contract implementation upgrades MUST be authorized only by an account holding `PROXY_UPGRADE_ROLE` in `_authorizeUpgrade`, and the logic implementation contract MUST have initializers disabled so it cannot be initialized by an attacker.

> Each ERC-7201 namespaced storage location (e.g. `ERC20BaseModuleStorageLocation`, `PauseModuleStorageLocation`, `CCIPModuleStorageLocation`, `SnapshotEngineModuleStorageLocation`) MUST be unique across the deep module inheritance chain so no two modules share a storage slot.

> When a `RuleEngine` is configured, every value-moving transfer MUST be validated by it (restriction code `0`) before balances change; the on-chain enforcement path MUST agree with the `detectTransferRestriction` view result.

> Only accounts holding `ERC20ENFORCER_ROLE` may execute `forcedTransfer` and partial-balance freeze/unfreeze (enforced by `onlyERC20Enforcer` / `onlyForcedTransferManager` in `ERC20EnforcementModule`).

> Only accounts holding `PAUSER_ROLE` may call `pause` / `unpause`, and only `onlyDeactivateContractManager` may call `deactivateContract` (enforced in `PauseModule`).

> Only `DEFAULT_ADMIN_ROLE` may `grantRole` / `revokeRole` for every other role; each role is administered by `DEFAULT_ADMIN_ROLE` (OpenZeppelin `AccessControlUpgradeable`).

> UUPS upgrades in `CMTATUpgradeableUUPS` are gated ONLY by `PROXY_UPGRADE_ROLE` with NO timelock or delay; there is no on-chain enforcement of a governance/timelock window before an implementation change takes effect.

> Initializers must run exactly once: `initialize` / `__*_init` are protected by `initializer` / `onlyInitializing`, and standalone (non-proxy) deployments call `_disableInitializers` in their constructor.

> Storage layout must be preserved across upgrades; all modules use ERC-7201 namespaced storage (`*StorageLocation` constants) to avoid slot collisions.

> Only the authorized token bridge (`CROSS_CHAIN_ROLE` / `onlyTokenBridge`, with `BURNER_FROM_ROLE` / `BURNER_SELF_ROLE` for burn paths) may call `crosschainMint` / `crosschainBurn` (IERC7802).

> When the contract is paused, no `transfer`, `transferFrom`, `mint`, or `burn` (non-forced) may succeed; the paused state is enforced in transfer validation (`ValidationModule` / `PauseModule`).

> Every transfer must pass configured validation: frozen (address or partial-balance) accounts cannot send/receive, non-allowlisted accounts are blocked (allowlist variant), and the `RuleEngine` `canTransfer` / `detectTransferRestriction` result must permit the transfer (rule-engine variant).

> A token holder cannot burn their own tokens; only addresses with the appropriate burner authority (`BURNER_ROLE`, `BURNER_FROM_ROLE`, `BURNER_SELF_ROLE`) or the `DEFAULT_ADMIN_ROLE` can destroy tokens (self-burn is forbidden by design).

> Once the contract is deactivated (`deactivated()` returns true), no `transfer`, `mint`, or `burn` operation can be performed.

> For any account, the active (unfrozen) balance `getActiveBalanceOf(account)` equals `balanceOf(account)` minus `getFrozenTokens(account)`.

> Tokens can only be removed from a frozen address through `forcedTransfer` (or `forcedBurn` in the Light version); standard `transfer`, `mint`, `burn`, `burnFrom`, and cross-chain mint/burn revert when the target/origin address is frozen.

> Every CMTAT balance-changing operation (`mint`, `burn`, `transfer`, `transferFrom`, `forcedTransfer`, `crosschainMint`, `crosschainBurn`, `transferAndCall`) MUST route through OpenZeppelin's `ERC20Upgradeable._update` override — OZ's `_balances` and `_totalSupply` are the single source of truth that all CMTAT modules (`SnapshotEngineModule`, `ERC20EnforcementModule`, `RuleEngine` checks) read from and depend on.

> All CMTAT authorization checks (`onlyMinter`, `onlyBurner`, `onlyERC20Enforcer`, `onlyEnforcer`, `onlyTokenBridge`, `PROXY_UPGRADE_ROLE`, etc.) MUST resolve exclusively through OpenZeppelin `AccessControlUpgradeable.hasRole` / `_checkRole`, with role state held in OZ's `AccessControlStorage` as the sole authority; no CMTAT module may maintain a parallel or cached authorization state.

> The ERC-7201 namespaced storage slots defined by CMTAT modules MUST NOT collide with the ERC-7201 storage slots used internally by OpenZeppelin upgradeable base contracts (`ERC20Storage`, `AccessControlStorage`, `PausableStorage`, `ERC2771ForwarderStorage`, and `Initializable`'s `_initialized` / `_initializing` slot); slot uniqueness MUST hold across the combined CMTAT + OZ inheritance chain, not just among CMTAT modules.

> Any replacement implementation authorized via `CMTATUpgradeableUUPS._authorizeUpgrade` MUST remain UUPS-compatible with OpenZeppelin `UUPSUpgradeable` (identical `proxiableUUID`, same ERC-1967 implementation slot) AND MUST preserve storage-layout compatibility with the OZ base contracts already deployed behind the proxy; upgrading the `openzeppelin-contracts-upgradeable` dependency to a storage-incompatible version constitutes a violation.

> During initialization, the CMTAT initializer MUST invoke every required OpenZeppelin base initializer (`__AccessControl_init`, `__Pausable_init`, `__ERC20_init`, `__UUPSUpgradeable_init`, and the ERC-2771 context setup) exactly once inside OZ `Initializable`'s `initializer` guard, and the logic implementation MUST call `_disableInitializers()` in its constructor; no OZ base state may be left uninitialized or be re-initializable by an attacker.

> All authorization checks throughout the modules MUST resolve the caller via `_msgSender()` (ERC-2771 aware) rather than raw `msg.sender`, so relayed meta-transactions attribute actions to the true signer, not the trusted forwarder.

> The `SnapshotEngine` balance-update hook MUST be invoked on every balance-changing operation before/consistently with the balance mutation, so recorded historical balances at any snapshot time equal the true balances at that time.

> When the allowlist is active, both `from` and `to` MUST be allowlisted for a transfer to succeed, while mint (`from == address(0)`) and burn (`to == address(0)`) edge cases MUST be handled without falsely requiring the zero address to be allowlisted.

> Only accounts holding `ENFORCER_ROLE` may freeze/unfreeze addresses (enforced by `onlyEnforcer` in `EnforcementModule`).

> Only the RuleEngine manager (`onlyRuleEngineManager`) may call `setRuleEngine`, and only privileged role holders may set the Snapshot/Document/Debt engines and CCIP admin (`onlyCCIPSetAdmin`).

> Cross-chain supply functions (`crosschainMint`, `crosschainBurn`, `burnFrom`, and the cross-chain `burn(uint256)`) revert when the contract is paused (`whenNotPaused`), unlike the core `mint` / `burn` which are allowed while paused.

> CMTAT's `_msgSender()`, `_msgData()`, and `_contextSuffixLength()` overrides MUST resolve, via the multiple-inheritance C3 linearization, to OpenZeppelin `ERC2771ContextUpgradeable` (not plain `ContextUpgradeable`), so that every authorization and value-moving path attributes actions to the true signer rather than the trusted forwarder; the override resolution spans the OZ base classes and the CMTAT module chain and MUST be consistent everywhere.

Security Assumptions:

- **Trusted Forwarder (ERC2771)** (Trusted)
  - Relay meta-transactions and append the real sender to calldata for _msgSender()
  - Must faithfully report the original signer; a malicious forwarder could spoof _msgSender and impersonate any account
- **CMTAT core/modules → openzeppelin-contracts-upgradeable (inheritance dependency)** (Trusted)
  - OZ `AccessControlUpgradeable` correctly enforces `hasRole` / `onlyRole` and stores role state as the sole authority for MINTER/BURNER/PAUSER/etc.
  - OZ `ERC20Upgradeable` owns `_balances` / `_totalSupply` and routes every balance change through the overridable `_update` hook that CMTAT extends
  - OZ `UUPSUpgradeable` manages the ERC-1967 implementation slot, `proxiableUUID`, and rollback protection; CMTAT only supplies `_authorizeUpgrade`
  - OZ `Initializable` enforces one-time initialization via the `initializer` / `reinitializer` guard
  - OZ `ERC2771ContextUpgradeable` / `ContextUpgradeable` resolve `_msgSender()` / `_msgData()` correctly for meta-transactions
  - OZ calls back into CMTAT's overrides (`_update`, `_msgSender`, `_authorizeUpgrade`, `supportsInterface`) via virtual dispatch
- **MINTER_ROLE** (SemiTrusted)
  - Mint new tokens via mint/mintBatch to specified recipients
  - Bounded to increasing supply of the protocol's own token (dilution risk, not direct seizure)
  - Cannot move or burn existing holder balances
- **PAUSER_ROLE** (SemiTrusted)
  - pause()/unpause() to halt all transfers, mints and burns
  - deactivateContract() to permanently disable the token (via onlyDeactivateContractManager)
  - Bounded: can brick/freeze operation of the protocol's OWN token but cannot redirect funds
- **ENFORCER_ROLE** (SemiTrusted)
  - Freeze/unfreeze (address-level) holders via EnforcementModule to block their transfers
  - Reversible compliance control; does not move or destroy funds
  - Bounded to the protocol's own holders
- **CROSS_CHAIN_ROLE (token bridge)** (SemiTrusted)
  - crosschainMint/crosschainBurn (IERC7802) used by an authorized bridge/TokenPool (e.g. Chainlink CCIP)
  - BURNER_FROM_ROLE / BURNER_SELF_ROLE control cross-chain burn paths
  - Bounded to bridge accounting of this token; a compromised bridge could inflate/deflate supply
- **ALLOWLIST_ROLE** (SemiTrusted)
  - Add/remove addresses from the allowlist controlling who may send/receive (AllowlistModule)
  - Bounded compliance gating; can block non-allowlisted holders from transferring but cannot seize funds
- **RULE_ENGINE_ROLE (RuleEngine manager)** (SemiTrusted)
  - setRuleEngine() to point transfer validation at an external IRuleEngine contract
  - Can set an arbitrary rule-engine address, which gates/allows all transfers
  - Bounded to transfer validation of this token; misconfiguration can brick or over-permit transfers
- **SNAPSHOOTER_ROLE / DOCUMENT_ROLE / EXTRA_INFORMATION_ROLE / DEBT_ROLE / DEBT_ENGINE_ROLE / CCIP admin** (SemiTrusted)
  - SNAPSHOOTER_ROLE: schedule/manage balance snapshots (SnapshotEngineModule)

    - DOCUMENT/EXTRA_INFORMATION_ROLE: manage documents, terms and token metadata
    - DEBT/DEBT_ENGINE_ROLE: set debt info and the external debt engine
    - CCIP admin: set the CCIP admin address (getCCIPAdmin)
    - All are low-impact metadata/config powers bounded to this token; none can move holder funds
- **External Engines (RuleEngine, SnapshotEngine, DocumentEngine, DebtEngine)** (SemiTrusted)
    - RuleEngine gates every transfer via canTransfer/detectTransferRestriction (can block transfers, must not revert unexpectedly)
    - SnapshotEngine invoked in _update to record balances
    - DocumentEngine/DebtEngine provide metadata/credit data
    - These are external contracts trusted to behave per interface; a malicious engine can DoS transfers but not directly seize balances
- **OZ library version / storage-layout compatibility across UUPS upgrades** (SemiTrusted)
    - New implementations are compiled against the same (or storage-compatible) OZ version so OZ base-contract storage (`ERC20Storage` , `AccessControlStorage` , `PausableStorage`) and CMTAT ERC-7201 namespaces remain layout-stable
    - The replacement implementation keeps `proxiableUUID` identical so OZ UUPS rollback protection accepts the upgrade
    - Inherited OZ initializers are not re-run in a way that resets base state during `reinitializer` upgrades
- **Users** (Untrusted)
    - Hold, transfer, approve tokens subject to RuleEngine/Allowlist/Pause/freeze checks
    - Call crosschainTransfer / ERC1363 transferAndCall
    - Cannot mint, burn others' tokens, pause, freeze, or manage roles
- **DEFAULT_ADMIN_ROLE** (Untrusted)
    - Grant/revoke ALL other roles via grantRole/revokeRole (AccessControlUpgradeable)
    - Can bootstrap itself into MINTER_ROLE, BURNER_ROLE, ERC20ENFORCER_ROLE, PROXY_UPGRADE_ROLE etc.
    - Effectively has unbounded control: can seize/destroy non-consenting holders' funds (via enforcer/burner) and (UUPS) replace the implementation
- **PROXY_UPGRADE_ROLE** (Untrusted)
    - Authorize UUPS implementation upgrades via _authorizeUpgrade in CMTATUpgradeableUUPS
    - Can replace the entire token logic with arbitrary code, redirecting/seizing all holder balances
- **ERC20ENFORCER_ROLE** (Untrusted)
    - Execute forcedTransfer to move tokens from ANY holder to another address without consent
    - Freeze/unfreeze a partial token balance of any holder
    - Intended as a regulatory/compliance seizure power in ERC-3643/ERC-7551 style
- **BURNER_ROLE** (Untrusted)
    - Burn tokens from any holder's balance via burn(account, value)/forcedBurn and burnBatch
    - Destroys non-consenting holders' tokens (compliance-driven but irreversible)
- **DEFAULT_ADMIN_ROLE / PROXY_UPGRADE_ROLE holder** (Untrusted)
    - Grant/revoke all roles via AccessControlUpgradeable
    - Authorize UUPS implementation upgrades via _authorizeUpgrade (no on-chain timelock)
    - Effective god-role over minting, burning, forced-transfer, and full contract replacement
- **CROSS_CHAIN_ROLE / BURNER_FROM_ROLE (token bridge)** (Untrusted)
    - Call crosschainMint / crosschainBurn (IERC7802) on the CMTAT token
    - Call burnFrom via BURNER_FROM_ROLE for cross-chain burn paths
    - Trusted to conserve supply parity across chains — a compromised bridge can mint unbacked tokens
- **ERC-2771 Trusted Forwarder (constructor address)** (Untrusted)
    - Relay meta-transactions and append true signer to calldata for _msgSender()
    - Immutable once deployed — baked into bytecode, cannot be corrected post-deployment
    - A malicious or buggy forwarder can spoof _msgSender() and impersonate any account across all role-gated functions
- **RuleEngine external contract** (Untrusted)

  - Gate every transfer via canTransfer / detectTransferRestriction
  - transferred() hook called during _update — a reverting engine bricks all transfers (DoS)
  - A malicious/misconfigured engine can block all transfers or allow non-compliant ones
- **Role distribution (MINTER/BURNER/ENFORCER/ERC20ENFORCER/PAUSER)** (Untrusted)
  - MINTER_ROLE: mint new supply to any address
  - BURNER_ROLE / BURNER_FROM_ROLE: destroy any holder's tokens
  - ERC20ENFORCER_ROLE: forced-transfer any holder's tokens to any address
  - PAUSER_ROLE: halt all transfers or permanently deactivate the contract
  - If all roles share the same key, a single compromise cascades to every privileged operation
- **CCIP admin (getCCIPAdmin) / cross-chain deployment address** (Untrusted)
  - getCCIPAdmin returns the address used by Chainlink CCIP TokenAdminRegistry for bridge authorization
  - setCCIPAdmin is gated by DEFAULT_ADMIN_ROLE — a compromised admin can redirect bridge admin
  - Address mismatch across chains can break CCIP token-admin registration and cross-chain mint/burn

2. Scope

Repository: CMTA/CMTAT
Commit: 49544f4
Branch: master

Scanned 41/41 units:

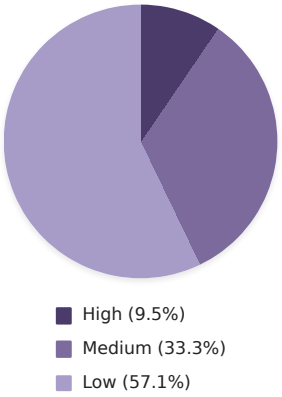
1. library ERC1404ExtendInterfaceId
2. library RuleEngineInterfaceId
3. function transferFrom
4. contract ValidationModuleAllowlist
5. function _burnOverride
6. function _mintOverride
7. function _update
8. function _canMintBurnByModule
9. function _burn
10. function _mintOverride
11. contract ValidationModuleRuleEngine
12. library EnforcementModuleLibrary
13. function _burnOverride
14. function _canTransferGenericByModule
15. contract CMTATBaseAccessControl
16. function _mintOverride
17. function _burnOverride
18. function transferFrom
19. contract CMTATBaseAllowlist
20. function transferFrom
21. function _checkActiveBalance
22. function transferFrom
23. function transferFrom
24. contract ERC20EnforcementModule
25. contract CMTATBaseRuleEngine
26. contract CMTATBaseCore
27. contract ERC20BaseModule
28. contract CMTATBaseERC1363
29. contract ERC20EnforcementModuleInternal
30. contract ValidationModule
31. contract CMTATBaseCommon
32. contract ERC20CrossChainModule
33. contract CMTATBaseERC20CrossChain
34. Cluster<CMTATBaseAllowlist,CMTATBaseCommon,CMTATBaseCore,ERC20BurnModule,ERC20EnforcementModuleInternal,ERC20MintModule,EnforcementModule,PauseModule,ValidationModule,ValidationModuleCore>
35. Cluster<CMTATBaseAllowlist,CMTATBaseCommon,CMTATBaseCore,CMTATBaseRuleEngine,ERC20BurnModule,ERC20EnforcementModuleInternal,ERC20MintModule,EnforcementModule,ValidationModule,ValidationModuleRuleEngine>
36. Cluster<CMTATBaseAllowlist,CMTATBaseCommon,CMTATBaseCore,CMTATBaseRuleEngine,ERC20EnforcementModuleInternal,EnforcementModule,PauseModule,ValidationModule,ValidationModuleCore,ValidationModuleRuleEngine>
37. Cluster<CMTATBaseCommon,CMTATBaseCore,CMTATBaseDebtEngine,CMTATBaseERC20CrossChain,CMTATBaseERC2771,CMTATUpgradeableDebtEngine,DebtEngineModule,ERC20BurnModule,ERC20BurnModuleInternal,ERC20CrossChainModule>
38. Cluster<CMTATBaseAllowlist,CMTATBaseCommon,CMTATBaseCore,CMTATBaseRuleEngine,ERC20BurnModule,ERC20EnforcementModuleInternal,ERC20MintModule,ValidationModule,ValidationModuleCore,ValidationModuleRuleEngine>
39. Cluster<CMTATBaseAllowlist,CMTATBaseERC1363,CMTATBaseERC2771,CMTATBaseERC7551,CMTATStandaloneERC1363,CMTATStandaloneERC7551,CMTATUpgradeableERC1363,CMTATUpgradeableERC7551,ERC2771Module,ERC7551Module>
40. Cluster<CMTATBaseAccessControl,CMTATBaseAllowlist,CMTATBaseCommon,CMTATBaseRuleEngine,ERC20BurnModule,ERC20EnforcementModuleInternal,ERC20MintModule,ValidationModule,ValidationModuleAllowlist,ValidationModuleRuleEngine>

41. Cluster<CMTATBaseAllowlist,CMTATBaseCommon,CMTATBaseCore,CMTATBaseRuleEngine,ERC20BaseModule,ERC20BurnModule,ERC20CrossChainModule,ERC20EnforcementModuleInternal,ERC20MintModule,ERC20MintModuleInternal>

3. Summary

Scan Started: 17 July 2026 01:44 UTC
Scan Completed: 17 July 2026 02:41 UTC
Report Generated: 17 July 2026 03:14 UTC

Severity	Count
High	2
Medium	7
Low	12
Total Issues	**21**



Severity Definitions:

- **High**: Vulnerabilities that enable direct theft or loss of user funds, protocol insolvency, or system-wide failures with realistic attack paths requiring minimal constraints.
- **Medium**: Vulnerabilities that either cause significant impact (fund loss, core functionality breaks) under constrained conditions, moderate impact (temporary DoS, localized losses) under realistic conditions, or minor impact with highly accessible exploit paths.
- **Low**: Vulnerabilities with minimal financial risk that affect non-core functionality, state handling, or minor logic under most conditions, or medium-impact issues requiring admin actions or extreme constraints to exploit.

4. True Positive Findings

4.1. High

4.1.1. Missing Deactivation Guard

HIGH

Entry Point: `CMTATBaseCore.forcedBurn`

Surfaced From: 2 units

Location: contracts/modules/0_CMTATBaseCore.sol:CMTATBaseCore.forcedBurn()

Touchpoints: `CMTATBaseCore.forcedBurn` (entry) → `CMTATBaseCore._burnOverride` (intermediate) → `ValidationModule._canMintBurnByModuleAndRevert` (intermediate) → `PauseModule.deactivateContract` (state-source) → `ERC20EnforcementModule.forcedTransfer` (entry) → `ERC20EnforcementModuleInternal._forcedTransfer` (sink) → `CMTATBaseCommon._update` (sink)

DESCRIPTION

forcedBurn can still reduce balances and totalSupply after the token has been deactivated. Normal burn paths route through _burnOverride(), which calls the validation module and rejects deactivated contracts, but forcedBurn deliberately skips that path and calls ERC20Upgradeable._burn() directly after only checking that the account is frozen. After pause() and deactivateContract(), a DEFAULT_ADMIN_ROLE holder can keep or set an account as frozen and destroy tokens despite the deactivation guarantee that burn operations stop permanently.

EVIDENCE

- `contracts/modules/0_CMTATBaseCore.sol:forcedBurn()` — burns with `ERC20Upgradeable._burn(account, value)` after only checking `EnforcementModule.isFrozen(account)`, with no `_requireNotDeactivated()` or validation-module call.
- `contracts/modules/0_CMTATBaseCore.sol:_burnOverride()` — the ordinary burn hook calls `ValidationModule._canMintBurnByModuleAndRevert(account)` before delegating to the ERC20 burn implementation, showing the missing counterpart in `forcedBurn()`.
- `contracts/modules/wrapper/controllers/ValidationModule.sol:_canMintBurnByModuleAndRevert()` — normal mint/burn validation explicitly calls `_requireNotDeactivated()` before allowing the supply change.
- `contracts/modules/wrapper/core/PauseModule.sol:deactivateContract()` — deactivation sets `_isDeactivated = true` after requiring the contract to be paused, making the later forced burn a deactivation-state bypass.
- `Invariant: deactivated token lifecycle` — once `deactivated()` is true, no transfer, mint, or burn operation should be possible, but `pause() -> deactivateContract() -> forcedBurn(frozenAccount, value, data)` still changes supply.
- `contracts/modules/wrapper/extensions/ERC20EnforcementModule.sol:ERC20EnforcementModule.forcedTransfer()` — both forced-transfer overloads are only role-gated and call `_forcedTransfer` without `whenNotPaused`, `_requireNotPaused`, or `_requireNotDeactivated`.
- `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:ERC20EnforcementModuleInternal._forcedTransfer()` — after unfreezing tokens, the sink calls `ERC20Upgradeable._transfer(from, to, value)` or `ERC20Upgradeable._burn(from, value)` when `to == address(0)`.
- `contracts/modules/wrapper/core/PauseModule.sol:PauseModule.deactivateContract()` — deactivation requires the contract to be paused, stores `_isDeactivated = true`, and `unpause()` rejects once deactivated, establishing a permanent shutdown state.
- `contracts/modules/wrapper/controllers/ValidationModule.sol:ValidationModule._canMintBurnByModuleAndRevert()` — normal mint/burn validation explicitly calls `_requireNotDeactivated`, showing the missing counterpart in forcedTransfer.
- `Invariant: deactivated lifecycle` — once `deactivated()` is true, no transfer, mint, or burn operation can be performed, but forcedTransfer can still transfer or burn balances.

PROOF OF CONCEPT

`test/poc/CMTATBaseCore_missing_deactivation_guard.js` — Deploys `CMTATStandaloneLight`, mints tokens, pauses and calls `deactivateContract()`, then shows that while the normal `burn()` path reverts with `EnforcedDeactivation`, `CMTATBaseCore.forcedBurn()` can still burn a frozen holder's tokens because it bypasses `_burnOverride()`. The PoC reproduces the issue as a true positive: the vulnerable implementation allows balance and `totalSupply` reduction after deactivation.

```
const { expect } = require('chai')
const { ethers } = require('hardhat')
const { loadFixture } = require('@nomicfoundation/hardhat-network-helpers')
const { deployCMTATLightStandalone } = require('../deploymentUtils')
```

```
// PoC for contracts/modules/0_CMTATBaseCore.sol:CMTATBaseCore.forcedBurn()
// CMTATStandaloneLight is the concrete deployment artifact that inherits CMTATBaseCore
// and exposes forcedBurn().
describe('CMTATBaseCore forcedBurn deactivation guard PoC', function () {
  const INITIAL_SUPPLY = 50n
  const BURN_AMOUNT = 20n
  const REASON = '0x'

  async function deployFixture () {
    const [deployer, admin, holder] = await ethers.getSigners()
    const cmtat = await deployCMTATLightStandalone(
      admin.address,
      deployer.address
    )
    await cmtat.waitForDeployment()

    return { cmtat, admin, holder }
  }

  it('should not allow forcedBurn to reduce balances or totalSupply after deactivateContract()', async function ()
{
    const { cmtat, admin, holder } = await loadFixture(deployFixture)

    // Arrange: mint tokens to a holder, then permanently deactivate the token.
    await cmtat.connect(admin).mint(holder.address, INITIAL_SUPPLY)
    expect(await cmtat.balanceOf(holder.address)).to.equal(INITIAL_SUPPLY)
    expect(await cmtat.totalSupply()).to.equal(INITIAL_SUPPLY)

    await cmtat.connect(admin).pause()
    await cmtat.connect(admin).deactivateContract()
    expect(await cmtat.paused()).to.equal(true)
    expect(await cmtat.deactivated()).to.equal(true)

    // Sanity check: the normal burn path routes through _burnOverride(), which
    // calls ValidationModule._canMintBurnByModuleAndRevert() and rejects a
    // deactivated contract.
    await expect(cmtat.connect(admin).burn(holder.address, 1n))
      .to.be.revertedWithCustomError(cmtat, 'EnforcedDeactivation')

    // The holder can still be frozen after deactivation; forcedBurn only checks
    // this frozen flag and then calls ERC20Upgradeable._burn() directly.
    await cmtat.connect(admin).setAddressFrozen(holder.address, true)
    expect(await cmtat.isFrozen(holder.address)).to.equal(true)

    // Security expectation: deactivation guarantees burn operations stop
    // permanently, so forcedBurn should also revert with EnforcedDeactivation.
    // On the vulnerable implementation this transaction succeeds, reducing both
    // holder balance and totalSupply, so this assertion fails and proves the bug.
    await expect(cmtat.connect(admin).forcedBurn(holder.address, BURN_AMOUNT, REASON))
      .to.be.revertedWithCustomError(cmtat, 'EnforcedDeactivation')

    // These post-conditions are reachable only on a fixed implementation.
    expect(await cmtat.balanceOf(holder.address)).to.equal(INITIAL_SUPPLY)
    expect(await cmtat.totalSupply()).to.equal(INITIAL_SUPPLY)
  })
})
```

4.1.2. Unchecked Overfrozen Balance Underflow

HIGH

Entry Point: `ERC20EnforcementModule.setFrozenTokens`

Surfaced From: 14 units

Location: contracts/modules/wrapper/extensions/ERC20EnforcementModule.sol:setFrozenTokens(); contracts/modules/internal/ERC20EnforcementModuleInternal.sol:_setFrozenTokens()/_getActiveBalanceOf()/_checkActiveBalance()

Touchpoints: `ERC20EnforcementModule.setFrozenTokens` (entry) → `ERC20EnforcementModuleInternal._setFrozenTokens` (state-source) → `ERC20EnforcementModuleInternal._checkActiveBalance` (sink) → `CMTATBaseCommon._mintOverride` (intermediate) → `CMTATBaseERC20CrossChain._mintOverride` (entry) → `CMTATBaseCommon._checkTransferred` (sink) → `ERC20EnforcementModuleInternal._setFrozenTokens` (intermediate) → `ERC20EnforcementModuleInternal._getActiveBalanceOf` (sink) → `CMTATBaseAccessControl._authorizeERC20Enforcer` (intermediate) → `ERC20EnforcementModuleInternal._setFrozenTokens` (sink) → `ERC20EnforcementModuleInternal._checkActiveBalance` (state-source) → `CMTATBaseCommon._checkTransferred` (intermediate) → `CMTATBaseAllowlist.canTransfer` (entry) → `CMTATBaseRuleEngine.canTransfer` (sink) → `ERC20CrossChainModule.crosschainMint` (entry) → `ERC20EnforcementModuleInternal._freezePartialTokens` (intermediate) → `ERC20MintModule.mint` (intermediate) → `ERC20CrossChainModule.crosschainMint` (intermediate) → `CMTATBaseCommon._mintOverride` (sink) → `ERC20EnforcementModuleInternal._unfreezeTokens` (sink)

DESCRIPTION

The active-balance logic assumes `frozenTokens(account) <= balanceOf(account)` and subtracts directly, but the exposed absolute `setFrozenTokens` / `_setFrozenTokens` path can write an arbitrary frozen amount without enforcing that cap, unlike bounded incremental freeze semantics. If an account is over-frozen, `_checkActiveBalance`, `_getActiveBalanceOf`, transfer/burn validation, forced-transfer recovery paths, and related `canTransfer` reads can revert by arithmetic underflow until a privileged correction. Setting a nonzero frozen amount for `address(0)` is especially damaging because mint validation treats `address(0)` as the source, so ordinary and cross-chain minting can be globally bricked. This violates frozen-balance accounting and lets the ERC20 enforcer role interfere with availability beyond normal partial-freeze semantics.

User Verdict: True Positive

Verdict Reason: Intercept demo: matches prior-scan TP

EVIDENCE

- `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:_checkActiveBalance()` — computes `ERC20Upgradeable.balanceOf(from) - frozenTokensLocal` without handling `frozenTokensLocal > balanceOf(from)`, so an over-frozen account triggers a Solidity arithmetic panic.
- `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:_setFrozenTokens()` — overwrites `$._frozenTokens[account]` with `value` without checking `value <= balanceOf(account)` or rejecting `account == address(0)`.
- `contracts/modules/wrapper/extensions/ERC20EnforcementModule.sol:setFrozenTokens()` — exposes the unchecked setter through the `onlyERC20Enforcer` entry point, so an enforcer can create the invalid frozen-accounting state.
- `contracts/modules/0_CMTATBaseCommon.sol:_mintOverride()` — passes `address(0)` as the `from` address into `_checkTransferred`, so a nonzero frozen amount recorded for the zero address makes minting hit `_checkActiveBalance` and revert.
- `[Invariant: active-balance accounting]` — the required relation `getActiveBalanceOf(account) == balanceOf(account) - getFrozenTokens(account)` is not safely maintained when frozen tokens exceed the balance, causing getters and transfer checks to underflow instead of reporting zero active balance.
- `contracts/modules/4_CMTATBaseERC20CrossChain.sol:_mintOverride()` — all mint and cross-chain mint paths in this contract are funneled into `CMTATBaseCommon._mintOverride(account, value)`.
- `contracts/modules/0_CMTATBaseCommon.sol:_mintOverride()` — the common mint validator calls `_checkTransferred(address(0), address(0), account, value)`, passing the zero address as the `from` account.
- `contracts/modules/0_CMTATBaseCommon.sol:_checkTransferred()` — the common transfer check unconditionally calls `ERC20EnforcementModuleInternal._checkActiveBalanceAndRevert(from, value)`, so minting is made dependent on address(0)'s frozen-token accounting.
- `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:_setFrozenTokens()` — the setter writes `_frozenTokens[account] = value` without a zero-address check or a `value <= balanceOf(account)` check, allowing a nonzero frozen amount to be assigned to address(0).
- `Trust assumption: ERC20ENFORCER_ROLE is Untrusted` — a compromised or malicious enforcer can trigger this state and block non-consenting minters and bridge mints until the frozen amount is cleared.
- `contracts/modules/wrapper/extensions/ERC20EnforcementModule.sol:setFrozenTokens()` — exposes an authorized overwrite of the frozen-token amount and forwards the caller-supplied `value` directly to `_setFrozenTokens(account, value)`.
- `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:_setFrozenTokens()` — writes `$._frozenTokens[account] = value` on the increase path without the `balanceOf(account) >= value` guard used by `_freezePartialTokens()`.

- `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:_getActiveBalanceOf()` — returns `balanceOf(account) - $._frozenTokens[account]`, so any over-frozen account makes the public `getActiveBalanceOf` view revert.
- `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:_checkActiveBalance()` — subtracts `frozenTokensLocal` from `balanceOf(from)` before returning a boolean, so pre-check views and transfer validation can panic instead of returning `false`.
- `Invariant: getActiveBalanceOf(account) equals balanceOf(account) minus getFrozenTokens(account)` — this invariant is not safely maintained because a reachable state makes the subtraction underflow rather than yielding a usable active-balance result.
- `contracts/modules/1_CMTATBaseAccessControl.sol:_authorizeERC20Enforcer()` — binds ERC20 enforcement storage writers to callers passing `onlyRole(ERC20ENFORCER_ROLE)` in CMTAT deployments.
- `contracts/modules/wrapper/extensions/ERC20EnforcementModule.sol:setFrozenTokens()` — exposes a public `onlyERC20Enforcer` overwrite path and forwards arbitrary `account` and `value` inputs to `_setFrozenTokens()`.
- `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:_setFrozenTokens()` — writes `_frozenTokens[account] = value` on the increase path without checking `value <= ERC20Upgradeable.balanceOf(account)`.
- `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:_freezePartialTokens()` — the sibling incremental-freeze path reads `balanceOf(account)` and requires the new frozen total not to exceed the balance, showing the missing counterpart guard in `setFrozenTokens`.
- `Invariant: getActiveBalanceOf(account) equals balanceOf(account) minus getFrozenTokens(account)` — over-frozen storage violates this invariant and makes subtracting readers/checks such as `_getActiveBalanceOf()` and `_checkActiveBalance()` revert or block value-moving operations.
- `contracts/modules/0_CMTATBaseCommon.sol:_mintOverride()` — passes `address(0)` as the `from` argument to `_checkTransferred` on every mint before calling `ERC20MintModuleInternal._mintOverride`.
- `contracts/modules/0_CMTATBaseCommon.sol:_checkTransferred()` — unconditionally checks `ERC20EnforcementModuleInternal._checkActiveBalanceAndRevert(from, value)`, so the mint sentinel `from == address(0)` is processed as an account balance.
- `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:_checkActiveBalance()` — when frozen tokens are nonzero, computes `ERC20Upgradeable.balanceOf(from) - frozenTokensLocal`, which underflows/reverts for `from == address(0)` after any positive zero-address frozen-token value is set.
- `contracts/modules/wrapper/extensions/ERC20EnforcementModule.sol:setFrozenTokens()` — exposes `_setFrozenTokens(account, value)` to `onlyERC20Enforcer` without rejecting `account == address(0)`.
- `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:_setFrozenTokens()` — writes the requested frozen amount directly and does not require `value <= balanceOf(account)`, allowing `frozenTokens[address(0)]` to become positive.
- `contracts/modules/wrapper/extensions/ERC20EnforcementModule.sol:setFrozenTokens()` — the public enforcement entry point lets an ERC20ENFORCER_ROLE holder overwrite an account's frozen token amount.
- `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:_setFrozenTokens()` — the storage write accepts `value` without checking `value <= ERC20Upgradeable.balanceOf(account)`, unlike `_freezePartialTokens()` which enforces that bound.
- `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:_getActiveBalanceOf()` and `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:_checkActiveBalance()` — both readers subtract frozen tokens from the live balance, so an overfrozen account underflows instead of producing a safe zero active balance or a false result.
- `contracts/modules/2_CMTATBaseAllowlist.sol:canTransfer()/canTransferFrom()` and `contracts/modules/0_CMTATBaseCommon.sol:_checkTransferred()` — the allowlist prediction and actual transfer/burn paths depend on `_checkActiveBalance`, so the corrupted frozen amount blocks normal value-moving operations.
- `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:_unfreezeTokens()` — the forcedTransfer recovery path also computes `balance - _frozenTokens[account]` before reducing frozen storage, so overfrozen accounts cannot be rescued through forcedTransfer.
- `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:_setFrozenTokens()` — the overwrite path stores `$._frozenTokens[account] = value` without checking `value <= balanceOf(account)` or rejecting `address(0)`.
- `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:_freezePartialTokens()` — the sibling freeze path requires `balance >= frozenTokensLocal`, showing the missing cap in `_setFrozenTokens` is not consistently enforced.
- `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:_checkActiveBalance()` — transfer validation computes `ERC20Upgradeable.balanceOf(from) - frozenTokensLocal` whenever frozen tokens are nonzero, which panics if the stored frozen amount exceeds the balance.
- `contracts/modules/0_CMTATBaseCommon.sol:_mintOverride()` — every mint calls `_checkTransferred(address(0), address(0), account, value)`, so over-freezing `address(0)` feeds the zero-address sentinel into the unsafe active-balance subtraction and bricks minting.
- `Invariant: getActiveBalanceOf(account) equals balanceOf(account) minus getFrozenTokens(account)` — unchecked over-freezing breaks the frozen-balance accounting invariant and turns reads/transfer checks into reverts instead of returning zero active balance.
- `contracts/modules/wrapper/extensions/ERC20EnforcementModule.sol:setFrozenTokens()` — exposes the absolute frozen-token overwrite through `onlyERC20Enforcer` and forwards unchecked `account` and `value` into `_setFrozenTokens`.

- `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:_setFrozenTokens()` — writes `$._frozenTokens[account] = value` on the increase path without reading `balanceOf(account)` or rejecting `address(0)` .
- `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:_freezePartialTokens()` — the sibling incremental-freeze path requires `balance >= frozenTokensLocal` , proving the missing cap in the overwrite path.
- `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:_checkActiveBalance()` — validation subtracts `frozenTokensLocal` from `ERC20Upgradeable.balanceOf(from)` whenever frozen tokens are positive, so over-frozen state reverts instead of producing an active balance.
- `contracts/modules/0_CMTATBaseCommon.sol:_mintOverride()` — mint validation calls `_checkTransferred(address(0), address(0), account, value)` , so a positive frozen amount on the zero address poisons every mint path that reaches this hook.
- `contracts/modules/wrapper/extensions/ERC20EnforcementModule.sol:setFrozenTokens()` — the public role-gated absolute setter exposes `_setFrozenTokens(account, value)` as a state-changing entry point.
- `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:_setFrozenTokens()` — the state source writes `$._frozenTokens[account] = value` without checking `value <= balanceOf(account)` , unlike the capped `_freezePartialTokens` sibling.
- `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:_checkActiveBalance()` — the sink computes `ERC20Upgradeable.balanceOf(from) - frozenTokensLocal` under the assumption that frozen tokens cannot exceed the balance.
- `contracts/modules/0_CMTATBaseCommon.sol:_mintOverride()` — mint validation passes `from == address(0)` into `_checkTransferred` , so over-freezing the zero address makes all common/allowlist mints revert by active-balance underflow.
- `Invariant: getActiveBalanceOf(account) equals balanceOf(account) minus getFrozenTokens(account)` — over-frozen storage breaks the invariant and turns the subtraction into a protocol-level DoS instead of a bounded freeze.
- `contracts/modules/wrapper/extensions/ERC20EnforcementModule.sol:setFrozenTokens()` — the role-gated public entry directly delegates the caller-supplied `value` to `_setFrozenTokens(account, value)` with no local balance check.
- `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:_setFrozenTokens()` — the sink assigns `$._frozenTokens[account] = value` on both increase and decrease paths without requiring `value <= ERC20Upgradeable.balanceOf(account)` .
- `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:_freezePartialTokens()` — the sibling partial-freeze path does enforce `balance >= frozenTokensLocal` , showing the missing counterpart check in `_setFrozenTokens` .
- `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:_getActiveBalanceOf()` — active balance is computed as `ERC20Upgradeable.balanceOf(account) - $._frozenTokens[account]` , which reverts if the unchecked frozen amount exceeds the balance.
- `Invariant: active balance accounting` — the project requires `getActiveBalanceOf(account)` to equal `balanceOf(account) - getFrozenTokens(account)` without underflow, which the unchecked exact-freeze write can violate.
- `contracts/modules/wrapper/extensions/ERC20EnforcementModule.sol:setFrozenTokens()` — the role-gated entry point accepts any `account` and forwards it to `_setFrozenTokens` without rejecting `address(0)` .
- `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:_setFrozenTokens()` — the storage writer overwrites `_frozenTokens[account]` with the requested value and never checks that the account is a real holder or has a nonzero balance.
- `contracts/modules/0_CMTATBaseCommon.sol:_mintOverride()` — every mint validates with `_checkTransferred(address(0), address(0), account, value)` before calling the ERC20 mint implementation.
- `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:_checkActiveBalance()` — when frozen tokens are nonzero, it computes `balanceOf(from) - frozenTokensLocal` , so a positive frozen amount on `address(0)` underflows/reverts.
- `Invariant: MINTER_ROLE/CROSS_CHAIN_ROLE supply authority` — minting and bridge supply restoration should not be globally disabled by the partial-freeze enforcement role.
- `contracts/modules/wrapper/extensions/ERC20EnforcementModule.sol:ERC20EnforcementModule.setFrozenTokens()` — exposes an absolute setter that forwards the caller-supplied amount to `_setFrozenTokens` without checking it against `balanceOf(account)` or rejecting `address(0)` .
- `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:ERC20EnforcementModuleInternal._setFrozenTokens()` — writes the new frozen amount directly, unlike `_freezePartialTokens` which enforces `balance >= frozenTokensLocal` before increasing the frozen balance.
- `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:ERC20EnforcementModuleInternal._checkActiveBalance()` — subtracts `frozenTokensLocal` from `ERC20Upgradeable.balanceOf(from)` and the comment states the code assumes frozen amounts cannot exceed balance.
- `contracts/modules/0_CMTATBaseCommon.sol:CMTATBaseCommon._mintOverride()` — routes mint validation through `_checkTransferred(address(0), address(0), account, value)` , so `frozenTokens[address(0)] > 0` reaches the unchecked active-balance subtraction before any mint.
- `Invariant: getActiveBalanceOf(account) == balanceOf(account) - getFrozenTokens(account)` — the exposed setter can create states where the subtraction underflows instead of returning a valid active balance.
- `contracts/modules/wrapper/extensions/ERC20EnforcementModule.sol:setFrozenTokens()` — exposes an absolute frozen-balance setter that delegates directly to `_setFrozenTokens` under onlyERC20Enforcer.

- `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:_setFrozenTokens()` — stores the requested `value` as `_frozenTokens[account]` without requiring `value <= balanceOf(account)`.
- `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:_checkActiveBalance()` — subtracts `ERC20Upgradeable.balanceOf(from) - frozenTokensLocal` after assuming frozen amounts cannot exceed the balance, so overfrozen state reverts instead of returning false.
- `contracts/modules/0_CMTATBaseCommon.sol:_checkTransferred()` — routes transfer, mint, burn, and minter-transfer validation through `_checkActiveBalanceAndRevert`, propagating the corrupted frozen-balance state into all value-moving paths.
- `Invariant: getActiveBalanceOf(account) equals balanceOf(account) minus getFrozenTokens(account)` — overfrozen writes violate the invariant and can make the active-balance getter and enforcement flows unusable.
- `contracts/modules/wrapper/extensions/ERC20EnforcementModule.sol:setFrozenTokens()` — the external role-gated entry forwards an arbitrary absolute frozen amount to `_setFrozenTokens`.
- `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:_setFrozenTokens()` — the setter writes `value` directly to `_frozenTokens[account]` and lacks the balance cap enforced by sibling `_freezePartialTokens()`.
- `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:_checkActiveBalance()` — active-balance validation subtracts `balanceOf(from) - frozenTokensLocal` under the stated assumption that frozen tokens cannot exceed balance.
- `contracts/modules/0_CMTATBaseCommon.sol:_mintOverride()` — mint validation calls `_checkTransferred(address(0), address(0), account, value)`, so over-freezing `address(0)` makes all CMTATBaseCommon-derived mint paths underflow before minting.
- `Invariant: getActiveBalanceOf(account) equals balanceOf(account) minus getFrozenTokens(account)` — the invariant is not safely maintained when the setter allows frozen tokens to exceed balance.

4.2. Medium

4.2.1. Overfrozen Balance Transfer Dos

MEDIUM

Unit Name: `transferFrom`

Location: contracts/modules/6_CMTATBaseERC1363.sol:CMTATBaseERC1363.transferFrom() via ERC20EnforcementModuleInternal._checkActiveBalance()

Touchpoints: `CMTATBaseERC1363.transferFrom` (entry) → `CMTATBaseCommon.transferFrom` (intermediate) → `ERC20EnforcementModuleInternal._checkActiveBalance` (sink) → `ERC20EnforcementModuleInternal._setFrozenTokens` (state-source)

DESCRIPTION

The transferFrom path assumes an account's frozen-token amount is never greater than its ERC20 balance, but the role-gated setFrozenTokens path can write an arbitrary frozen amount without a balance cap. When transferFrom later validates that account, _checkActiveBalance subtracts frozenTokens from balanceOf and can underflow/revert before the ERC20 transfer, bricking delegated transfers and active-balance checks for the over-frozen account until privileged state repair. This violates the active-balance invariant and lets the partial-freeze authority create a stronger, error-prone lock state than the capped partial-freeze path allows.

User Verdict: True Positive

Verdict Reason: Intercept demo: matches prior-scan TP

EVIDENCE

- `contracts/modules/6_CMTATBaseERC1363.sol:transferFrom()` — the scanned entry point delegates every delegated transfer into the common CMTAT transfer validation path.
- `contracts/modules/0_CMTATBaseCommon.sol:transferFrom()` — delegated transfers call `_checkTransferred(_msgSender(), from, to, value)` before reaching the ERC20 allowance and transfer logic.
- `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:_checkActiveBalance()` — when `frozenTokensLocal > 0`, the active-balance calculation directly computes `balanceOf(from) - frozenTokensLocal` under the unguarded assumption that frozen tokens cannot exceed balance.
- `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:_setFrozenTokens()` — the overwrite path stores `value` as `_frozenTokens[account]` without checking `value <= balanceOf(account)`, while the sibling `_freezePartialTokens()` path does enforce that cap.
- `Invariant: active balance accounting` — `getActiveBalanceOf(account)` and transfer eligibility must equal `balanceOf(account) - getFrozenTokens(account)`, which is broken once a role holder sets frozen tokens above the actual balance.

4.2.2. Unchecked Overfrozen Balance

MEDIUM

Entry Point: `ERC20EnforcementModule.setFrozenTokens`

Location: contracts/modules/wrapper/extensions/ERC20EnforcementModule.sol:ERC20EnforcementModule.setFrozenTokens()

Touchpoints: `ERC20EnforcementModule.setFrozenTokens` (entry) → `ERC20EnforcementModuleInternal._setFrozenTokens` (state-source) → `ERC20MintModule.mint` (intermediate) → `CMTATBaseCommon._mintOverride` (intermediate) → `ERC20EnforcementModuleInternal._checkActiveBalance` (sink)

DESCRIPTION

setFrozenTokens writes an absolute frozen-token amount without capping it to the account balance or excluding address(0). CMTATBaseCommon-derived mint paths treat address(0) as the source and run the active-balance check before minting, so an ERC20ENFORCER_ROLE holder can set frozenTokens[address(0)] to any nonzero value and make every subsequent mint underflow when _checkActiveBalance subtracts it from balanceOf(address(0)). The same unchecked setter can over-freeze a real holder, causing getActiveBalanceOf, transfer, burn, and forced-transfer recovery paths to revert until a privileged account repairs the frozen amount, breaking the active-balance accounting invariant.

User Verdict: True Positive

Verdict Reason: Intercept demo: matches prior-scan TP

EVIDENCE

- `contracts/modules/wrapper/extensions/ERC20EnforcementModule.sol:setFrozenTokens()` — the public absolute-freeze entry forwards value to _setFrozenTokens without a balance cap or zero-address guard.
- `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:_setFrozenTokens()` — the state writer stores the new frozen amount directly, unlike _freezePartialTokens which checks balance >= frozenTokensLocal.
- `contracts/modules/0_CMTATBaseCommon.sol:_mintOverride()` — every full-module mint calls _checkTransferred(address(0), address(0), account, value) before executing the ERC20 mint.
- `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:_checkActiveBalance()` — when frozenTokensLocal > 0, the sink subtracts frozenTokensLocal from ERC20Upgradeable.balanceOf(from) and will underflow for address(0) or any over-frozen account.
- `Invariant: getActiveBalanceOf(account) equals balanceOf(account) minus getFrozenTokens(account)` — the unchecked setter can create frozenTokens(account) > balanceOf(account), making that invariant unrepresentable and bricking dependent paths.

4.2.3. Unchecked Frozen Token Cap

MEDIUM

Entry Point: `ERC20EnforcementModule.setFrozenTokens`

Location: contracts/modules/wrapper/extensions/ERC20EnforcementModule.sol:ERC20EnforcementModule.setFrozenTokens()

Touchpoints: `ERC20EnforcementModule.setFrozenTokens` (entry) → `ERC20EnforcementModuleInternal._setFrozenTokens` (state-source) → `CMTATBaseCommon._mintOverride` (intermediate) → `CMTATBaseCommon._checkTransferred` (intermediate) → `ERC20EnforcementModuleInternal._checkActiveBalance` (sink)

DESCRIPTION

The absolute partial-freeze setter writes an arbitrary frozen-token amount without enforcing frozenTokens(account) <= balanceOf(account). Once an ERC20ENFORCER_ROLE holder over-freezes an account, the shared active-balance readers subtract the excessive frozen amount from the ERC20 balance and revert by underflow, bricking transfers, burns, getActiveBalanceOf(), and forced-transfer recovery for that account until privileged repair. The cross-module impact is worse for address(0): CMTATBaseCommon uses address(0) as the mint source and runs the active-balance check before minting, so setFrozenTokens(address(0), 1) can make all mint paths in common-derived deployments revert even though the zero address has no balance to freeze.

User Verdict: True Positive

Verdict Reason: Intercept demo: matches prior-scan TP

EVIDENCE

- `contracts/modules/wrapper/extensions/ERC20EnforcementModule.sol:setFrozenTokens()` — The public role-gated entry forwards the caller-supplied absolute `value` to `_setFrozenTokens` with no balance cap or zero-address rejection.
- `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:_setFrozenTokens()` — The state writer assigns `$._frozenTokens[account] = value` on the freeze path without checking `value <= ERC20Upgradeable.balanceOf(account)`.
- `contracts/modules/internal/ERC20EnforcementModuleInternal.sol:_checkActiveBalance()` — The validation sink computes `ERC20Upgradeable.balanceOf(from) - frozenTokensLocal` whenever frozen tokens are nonzero, so an over-frozen account causes arithmetic underflow instead of returning a bounded active balance.
- `contracts/modules/0_CMTATBaseCommon.sol:_mintOverride()` — Minting calls `_checkTransferred(address(0), address(0), account, value)`, causing a nonzero frozen amount on address(0) to be read as the mint source and to brick mints.
- `Invariant: getActiveBalanceOf(account) equals balanceOf(account) minus getFrozenTokens(account)` — The unchecked setter allows frozen tokens to exceed the only ERC20 balance source of truth, making the active-balance invariant unrepresentable and reverting reads.

4.2.4. Burn Operator Context Bypass

MEDIUM

Entry Point: `ERC20CrossChainModule.burnFrom`

Location: contracts/modules/wrapper/options/ERC20CrossChainModule.sol:ERC20CrossChainModule.burnFrom()

Touchpoints: `ERC20CrossChainModule.burnFrom` (entry) → `ERC20CrossChainModule._burnFrom` (intermediate) → `ERC20CrossChainModule._burn` (sink) → `CMTATBaseERC20CrossChain._burnOverride` (intermediate) → `CMTATBaseCommon._burnOverride` (sink) → `ValidationModuleRuleEngine._transferred` (sink)

DESCRIPTION

The cross-chain burn-from path captures the actual burner/spender but drops it before running the concrete transfer-validation pipeline. `burnFrom` spends the caller's allowance and passes `sender` through `_burnFrom` into `_burn`, but `_burn` invokes `_burnOverride(account, value)` without the sender. In the CMTATBaseERC20CrossChain inheritance chain this reaches `CMTATBaseCommon._burnOverride`, which hardcodes `spender = address(0)` for `_checkTransferred`; the RuleEngine therefore receives the no-operator `transferred(from,to,value)` hook rather than the spender-aware hook. A `BURNER_FROM_ROLE` holder can burn from an approved account even when spender-specific compliance, frozen-spender checks, or `canTransferFrom` / `detectTransferRestrictionFrom` would reject that burner.

EVIDENCE

- `contracts/modules/wrapper/options/ERC20CrossChainModule.sol:burnFrom()` — the entry records `address sender = _msgSender()` and passes it to `_burnFrom`, proving the actual burn operator is known at the public boundary.
- `contracts/modules/wrapper/options/ERC20CrossChainModule.sol:_burnFrom()` — the operator's allowance is spent and `_burn(sender, account, value)` is called, so the flow is explicitly a spender-mediated burn.
- `contracts/modules/wrapper/options/ERC20CrossChainModule.sol:_burn()` — the sink discards `sender` for validation and calls `_burnOverride(account, value)` with only the token owner and amount.
- `contracts/modules/0_CMTATBaseCommon.sol:_burnOverride()` — the concrete validation path hardcodes `_checkTransferred(address(0), account, address(0), value)`, preventing spender-specific checks from running on burns.
- `contracts/modules/wrapper/extensions/ValidationModule/ValidationModuleRuleEngine.sol:_transferred()` — the RuleEngine dispatches to the spender-aware hook only when `spender != address(0)`, so the dropped operator changes the compliance path used by the state-changing burn.

PROOF OF CONCEPT

`test/poc/cluster_58aaaa3a_burn_operator_context_bypass.js` — Deploys a mock RuleEngine that rejects the actual burner as a spender, verifies `transferFrom` is rejected, then attempts `ERC20CrossChainModule.burnFrom()` to show the burn path drops spender/operator context before validation. The PoC confirms the issue as a true positive because the vulnerable `burnFrom` call succeeds where spender-aware compliance checks should revert.

```
const { expect } = require('chai')
const { ethers } = require('hardhat')
const {
  deployCMTATStandalone,
  fixture,
  loadFixture
} = require('../deploymentUtils')

const VALUE = 10n

describe('PoC - burnFrom drops operator context before RuleEngine validation', function () {
  async function deployFixture () {
    const accounts = await loadFixture(fixture)
    const cmtat = await deployCMTATStandalone(
      accounts._.address,
      accounts.admin.address,
      accounts.deployerAddress.address
    )

    const holder = accounts.address2
    const recipient = accounts.address3
    const authorizedSpender = accounts.address1
    const unauthorizedBurner = accounts.attacker

    // RuleEngineMock rejects canTransferFrom/transferred(spender, ...) for every
```

```
    // non-zero spender except `authorizedSpender`, while allowing the
    // no-operator canTransfer/transferred(from, ...) path for VALUE.
    const RuleEngineMock = await ethers.getContractFactory('RuleEngineMock')
    const ruleEngine = await RuleEngineMock.deploy(authorizedSpender.address)
    await ruleEngine.waitForDeployment()

    await cmtat.connect(accounts.admin).setRuleEngine(await ruleEngine.getAddress())

    const burnerFromRole = await cmtat.BURNER_FROM_ROLE()
    await cmtat.connect(accounts.admin).grantRole(burnerFromRole, unauthorizedBurner.address)

    await cmtat.connect(accounts.admin).mint(holder.address, VALUE)
    await cmtat.connect(holder).approve(unauthorizedBurner.address, VALUE)

    return {
      cmtat,
      ruleEngine,
      holder,
      recipient,
      authorizedSpender,
      unauthorizedBurner
    }
  }

  it('should reject burnFrom when the RuleEngine rejects the actual burner/spender', async function () {
    const {
      cmtat,
      ruleEngine,
      holder,
      recipient,
      unauthorizedBurner
    } = await loadFixture(deployFixture)

    // Sanity check: the configured RuleEngine explicitly rejects this operator.
    expect(
      await ruleEngine.canTransferFrom(
        unauthorizedBurner.address,
        holder.address,
        ethers.ZeroAddress,
        VALUE
      )
    ).to.equal(false)

    // The token's spender-aware pre-check also reports that this burn-from
    // should be non-compliant.
    expect(
      await cmtat.canTransferFrom(
        unauthorizedBurner.address,
        holder.address,
        ethers.ZeroAddress,
        VALUE
      )
    ).to.equal(false)

    // Sibling path check: ordinary transferFrom carries _msgSender() into the
    // validation pipeline, so the same unauthorized operator is rejected.
    await expect(
      cmtat
        .connect(unauthorizedBurner)
        .transferFrom(holder.address, recipient.address, VALUE)
    )
      .to.be.revertedWithCustomError(ruleEngine, 'RuleEngine_InvalidTransfer')
      .withArgs(holder.address, recipient.address, VALUE)

    // Security expectation: burnFrom must be validated with the real spender as
    // well. On vulnerable code this DOES NOT revert: burnFrom spends the
    // allowance and CMTATBaseCommon._burnOverride calls _checkTransferred with
    // spender = address(0), selecting RuleEngine.transferred(from, to, value)
    // instead of transferred(spender, from, to, value).
    await expect(
      cmtat.connect(unauthorizedBurner).burnFrom(holder.address, VALUE)
    )
      .to.be.revertedWithCustomError(ruleEngine, 'RuleEngine_InvalidTransfer')
      .withArgs(holder.address, ethers.ZeroAddress, VALUE)
  })
```

```
})
```

4.2.5. Missing Zero Address Validation

MEDIUM

Entry Point: `EnforcementModule.batchSetAddressFrozen`

Surfaced From: 11 units

Location: contracts/modules/internal/common/EnforcementModuleLibrary.sol:_checkInput()

Touchpoints: `EnforcementModule.batchSetAddressFrozen` (entry) → `EnforcementModuleInternal._addAddressesToTheList` (intermediate) → `CMTATBaseCore.transfer` (entry) → `ValidationModule._canTransferisFrozen` (sink) → `EnforcementModule.setAddressFrozen` (entry) → `CMTATBaseAllowlist._authorizeFreeze` (intermediate) → `CMTATBaseCommon.transfer` (entry) → `CMTATBaseAllowlist._checkTransferred` (intermediate) → `ValidationModule._canTransferisFrozenAndRevert` (sink) → `CMTATBaseCore._minterTransferOverride` (entry) → `EnforcementModule.setAddressFrozen` (state-source) → `CMTATBaseERC20CrossChain.transfer` (entry) → `CMTATBaseCommon.transfer` (intermediate) → `ValidationModuleERC1404.detectTransferRestriction` (state-source) → `EnforcementModuleInternal._addAddressToTheList` (state-source) → `CMTATBaseCore.transfer` (intermediate)

DESCRIPTION

The enforcement address-freeze paths allow address(0) to be written into the frozen-address list instead of rejecting it as an invalid account. This can occur through the batch path whose shared input validator only checks array length, and also through the single setAddressFrozen path authorized by ENFORCER_ROLE. Because direct-transfer style paths, including standard transfers, ERC1363 transferAndCall, and minter transfer/batch-transfer flows, use address(0) as a synthetic spender/sentinel, and the validation layer treats a frozen spender/account as a transfer blocker rather than ignoring the sentinel, freezing address(0) can globally disable otherwise-valid direct transfers without using the PAUSER_ROLE-controlled pause mechanism; delegated transferFrom calls with a nonzero spender do not hit the same sentinel, creating an inconsistent pause-like effect limited to direct-transfer flows. ERC-1404 restriction views that do not check the same sentinel can still report no restriction, creating a view/enforcement mismatch. Zero-address account entries should be rejected before frozen-list updates, or transfer validation should avoid treating the sentinel as a real frozen account.

EVIDENCE

- `contracts/modules/internal/common/EnforcementModuleLibrary.sol:_checkInput()` — The helper requires only non-empty equal-length arrays and never rejects `accounts[i] == address(0)`.
- `contracts/modules/internal/EnforcementModuleInternal.sol:_addAddressesToTheList()` — Batch enforcement writes every validated account/status pair into the frozen-address list after the helper returns.
- `contracts/modules/wrapper/core/EnforcementModule.sol:batchSetAddressFrozen()` — The role-gated public batch freeze entry point reaches the unchecked helper and can set `address(0)` frozen.
- `contracts/modules/0_CMTATBaseCore.sol:transfer()` — Direct transfers call validation with `address(0)` as the spender argument.
- `contracts/modules/wrapper/controllers/ValidationModule.sol:_canTransferisFrozen()` — Validation blocks standard transfers when `EnforcementModule.isFrozen(spender)` is true, so a frozen zero-address sentinel bricks direct transfers globally.
- `contracts/modules/0_CMTATBaseCommon.sol:transfer()` — direct transfers pass `address(0)` as the spender sentinel into `_checkTransferred` before updating balances.
- `contracts/modules/2_CMTATBaseAllowlist.sol:_checkTransferred()` — the allowlist base forwards that sentinel spender to `ValidationModule._canTransferGenericByModuleAndRevert` for standard-transfer validation.
- `contracts/modules/wrapper/controllers/ValidationModule.sol:_canTransferisFrozenAndRevert()` — the frozen-address check calls `EnforcementModule.isFrozen(spender)` without excluding `address(0)`, so a frozen sentinel makes the transfer path revert.
- `contracts/modules/wrapper/core/EnforcementModule.sol:setAddressFrozen()` and `contracts/modules/2_CMTATBaseAllowlist.sol:_authorizeFreeze()` — ENFORCER_ROLE can set any account's frozen flag and there is no zero-address guard before the storage write.
- `contracts/modules/wrapper/controllers/ValidationModuleAllowlist.sol:_canTransferStandardByModuleAllowlist()` — the allowlist sibling explicitly skips `spender == address(0)`, showing the missing sentinel guard in the freeze check.
- `contracts/modules/0_CMTATBaseCore.sol:transfer()` — ordinary direct transfers call `ValidationModule._canTransferGenericByModuleAndRevert(address(0), from, to)` before moving tokens.
- `contracts/modules/0_CMTATBaseCore.sol:_minterTransferOverride()` — the minter batch-transfer path also passes `address(0)` as the validation spender, so it is affected by the same poisoned sentinel state.
- `contracts/modules/wrapper/controllers/ValidationModule.sol:_canTransferisFrozenAndRevert()` — the validation code reverts when `EnforcementModule.isFrozen(spender)` is true and does not exempt `spender == address(0)`.
- `contracts/modules/wrapper/core/EnforcementModule.sol:setAddressFrozen()` — the enforcer-facing freeze entry point accepts an arbitrary `account` and delegates the write without a zero-address rejection.

- `Invariant: PAUSER_ROLE separation` — a global transfer halt is supposed to be controlled by `pause()`, but freezing the zero-spender sentinel lets ENFORCER_ROLE halt direct transfers with a single freeze write.
- `contracts/modules/wrapper/core/EnforcementModule.sol:setAddressFrozen()` — writes the supplied `account` to the frozen-address list through `_addAddressToTheList` without rejecting `address(0)`.
- `contracts/modules/4_CMTATBaseERC20CrossChain.sol:transfer()` — delegates every direct transfer to `CMTATBaseCommon.transfer`, so the target contract inherits the common sentinel-spender validation path.
- `contracts/modules/0_CMTATBaseCommon.sol:transfer()` — calls `_checkTransferred(address(0), from, to, value)`, using `address(0)` as the no-spender sentinel for direct transfers.
- `contracts/modules/wrapper/controllers/ValidationModule.sol:_canTransferisFrozenAndRevert()` — reverts when `EnforcementModule.isFrozen(spender)` is true before considering the real sender and recipient.
- `contracts/modules/wrapper/extensions/ValidationModule/ValidationModuleERC1404.sol:detectTransferRestriction()` — checks pause, deactivation, `from`, and `to` but not the sentinel spender, so ERC-1404 preflight can return OK while `transfer` reverts.
- `contracts/modules/wrapper/core/EnforcementModule.sol:setAddressFrozen()` — the role-gated freeze entry accepts any `account` and forwards it to the freeze list without rejecting `address(0)`.
- `contracts/modules/0_CMTATBaseCore.sol:transfer()` — the core direct-transfer path passes `address(0)` as the spender argument to `ValidationModule._canTransferGenericByModuleAndRevert`.
- `contracts/modules/0_CMTATBaseCommon.sol:transfer()` — the common/allowlist direct-transfer path uses the same `address(0)` spender sentinel before mutating balances.
- `contracts/modules/wrapper/controllers/ValidationModule.sol:_canTransferisFrozenAndRevert()` — the sink checks `EnforcementModule.isFrozen(spender)` first and reverts when the zero-address sentinel has been frozen.
- `Invariant: only PAUSER_ROLE may pause transfers` — freezing the sentinel gives `ENFORCER_ROLE` a pause-like global direct-transfer halt that is not routed through `PauseModule.pause()`.
- `contracts/modules/wrapper/core/EnforcementModule.sol:setAddressFrozen()` — the role-gated freeze entry accepts an arbitrary account and does not reject address(0).
- `contracts/modules/internal/EnforcementModuleInternal.sol:_addAddressToTheList()` — the internal state writer stores the freeze status for whatever account is supplied, including the zero address.
- `contracts/modules/0_CMTATBaseCore.sol:transfer()` — direct transfers pass address(0) as the spender argument to ValidationModule._canTransferGenericByModuleAndRevert.
- `contracts/modules/0_CMTATBaseCommon.sol:transfer()` — full-module direct transfers also call _checkTransferred with address(0) as the spender sentinel.
- `contracts/modules/wrapper/controllers/ValidationModule.sol:_canTransferisFrozenAndRevert()` — the validation sink checks EnforcementModule.isFrozen(spender) before from/to and reverts when the zero-address sentinel has been frozen.
- `contracts/modules/wrapper/core/EnforcementModule.sol:setAddressFrozen()` — The role-gated freeze entry accepts any `account` and forwards it to the internal list without rejecting `address(0)`.
- `contracts/modules/internal/EnforcementModuleInternal.sol:_addAddressToTheList()` — The sink stores the supplied address directly in `_list[account]`, so the zero address can become frozen state.
- `contracts/modules/0_CMTATBaseCore.sol:transfer()` — Direct transfers pass `address(0)` as the spender argument to the validation module rather than the actual caller.
- `contracts/modules/wrapper/controllers/ValidationModule.sol:_canTransferisFrozenAndRevert()` — Standard-transfer validation treats the spender argument as a real account and reverts when `EnforcementModule.isFrozen(spender)` is true.
- `Invariant: Only PAUSER_ROLE may pause/unpause standard transfers` — Freezing the zero-address sentinel gives ENFORCER_ROLE a pause-like effect on direct transfers without using PauseModule.
- `contracts/modules/wrapper/core/EnforcementModule.sol:setAddressFrozen()` — accepts the supplied `account` and forwards it to the freeze list without excluding `address(0)`.
- `contracts/modules/0_CMTATBaseCommon.sol:transfer()` — passes `address(0)` as the `spender` sentinel into `_checkTransferred` for ordinary direct transfers.
- `contracts/modules/wrapper/controllers/ValidationModule.sol:_canTransferisFrozenAndRevert()` — reverts when `EnforcementModule.isFrozen(spender)` is true, so a frozen zero sentinel blocks standard transfers.
- `contracts/modules/wrapper/controllers/ValidationModuleAllowlist.sol:_canTransferStandardByModuleAllowlist()` — explicitly ignores `spender == address(0)` for allowlist checks, showing the sentinel is not meant to be treated as a real transacting account.
- `contracts/modules/wrapper/core/EnforcementModule.sol:EnforcementModule.setAddressFrozen()` — accepts any `account` value and forwards it to `_addAddressToTheList` without rejecting `address(0)`.
- `contracts/modules/internal/EnforcementModuleInternal.sol:EnforcementModuleInternal._addAddressToTheList()` — stores the frozen status for the supplied address directly, so the zero address can be marked frozen.
- `contracts/modules/0_CMTATBaseCore.sol:CMTATBaseCore.transfer()` — validates direct transfers with `ValidationModule._canTransferGenericByModuleAndRevert(address(0), from, to)`, using the zero address as a spender sentinel.
- `contracts/modules/0_CMTATBaseCommon.sol:CMTATBaseCommon.transfer()` — the Common/Allowlist/RuleEngine transfer path likewise calls `_checkTransferred(address(0), from, to, value)` before the ERC20 transfer.

- `contracts/modules/wrapper/controllers/ValidationModule.sol:ValidationModule._canTransferisFrozenAndRevert()` — checks `EnforcementModule.isFrozen(spender)` before checking `from` or `to`, so a frozen zero-address sentinel reverts all direct transfers even though the zero address is not a participant.
- `contracts/modules/wrapper/core/EnforcementModule.sol:setAddressFrozen()` — the role-gated freeze entry point accepts any `account` and stores the status without rejecting `address(0)`.
- `contracts/modules/0_CMTATBaseCommon.sol:transfer()` — direct transfers deliberately pass `address(0)` as the `spender` argument into `_checkTransferred`.
- `contracts/modules/wrapper/controllers/ValidationModule.sol:_canTransferisFrozenAndRevert()` — standard-transfer validation checks `EnforcementModule.isFrozen(spender)` before `from` or `to` and has no `spender != address(0)` guard.
- `contracts/modules/wrapper/controllers/ValidationModuleAllowlist.sol:_canTransferStandardByModuleAllowlist()` — the allowlist sibling explicitly ignores a zero spender, showing the missing counterpart guard in the freeze check.
- `Invariant: PAUSER_ROLE transfer circuit breaker` — global transfer halts should be controlled by pause/deactivation, not by freezing the zero-address sentinel through ENFORCER_ROLE.
- `contracts/modules/wrapper/core/EnforcementModule.sol:setAddressFrozen()` — the role-gated freeze entry accepts an arbitrary `account` and forwards it to the freeze list without rejecting `address(0)`.
- `contracts/modules/internal/EnforcementModuleInternal.sol:_addAddressToTheList()` — the freeze state is written for the exact supplied address, so `address(0)` can become frozen state.
- `contracts/modules/0_CMTATBaseCore.sol:transfer()` and `contracts/modules/0_CMTATBaseCommon.sol:transfer()` — direct transfers pass `address(0)` as the spender/no-spender sentinel into the validation path.
- `contracts/modules/wrapper/controllers/ValidationModule.sol:_canTransferisFrozenAndRevert()` — validation rejects when `EnforcementModule.isFrozen(spender)` is true, so a frozen zero sentinel blocks every direct transfer.
- `Trust boundary: ENFORCER_ROLE is bounded to account freezing, not global pause` — freezing the sentinel gives that role a PAUSER-like global availability impact.

PROOF OF CONCEPT

`test/poc/EnforcementModuleLibrary_missing_zero_address_validation.js` demonstrates that the batch freeze path accepts `address(0)` via `batchSetAddressFrozen`, causing `transfer()` to treat the frozen zero-address spender sentinel as a global blocker while `paused()` remains false. The PoC reproduces the issue, confirming `contracts/modules/internal/common/EnforcementModuleLibrary.sol:_checkInput()` missing zero-address validation is a true positive.

```
const { expect } = require('chai')
const { ethers } = require('hardhat')
const {
  deployCMTATLightStandalone,
  fixture,
  loadFixture
} = require('../deploymentUtils')

// PoC for contracts/modules/internal/common/EnforcementModuleLibrary.sol:_checkInput().
// The vulnerable batch freeze path accepts address(0). In CMTATBaseCore.transfer(),
// direct ERC20 transfers validate with address(0) as the spender sentinel, so a frozen
// zero address makes every direct transfer fail even though the token is not paused.
describe('PoC - batch freeze of address(0) disables direct transfers', function () {
  async function deployFixture () {
    const ctx = await fixture()
    const cmtat = await deployCMTATLightStandalone(
      ctx.admin.address,
      ctx.deployerAddress.address
    )
    return { ...ctx, cmtat }
  }

  it('does not let ENFORCER_ROLE globally block direct transfers by freezing address(0)', async function () {
    const { cmtat, admin, address1, address2 } = await loadFixture(deployFixture)

    // Arrange: admin is the default admin and is implicitly authorized to mint/freeze.
    // A normal holder-to-holder direct transfer succeeds before the zero-address freeze.
    await cmtat.connect(admin).mint(address1.address, 100n)
    await expect(cmtat.connect(address1).transfer(address2.address, 1n))
      .to.emit(cmtat, 'Transfer')
      .withArgs(address1.address, address2.address, 1n)

    // Act: attempt the exact vulnerable batch path. A fixed implementation should
    // reject address(0) here; if so, direct transfers remain available and the test passes.
    let zeroAddressFreezeAccepted = false
    try {
      await cmtat
```

```
        .connect(admin)
        .batchSetAddressFrozen([ethers.ZeroAddress], [true])
      zeroAddressFreezeAccepted = true
    } catch (e) {
      zeroAddressFreezeAccepted = false
    }

    if (!zeroAddressFreezeAccepted) {
      expect(await cmtat.isFrozen(ethers.ZeroAddress)).to.equal(false)
      await expect(cmtat.connect(address1).transfer(address2.address, 1n))
        .to.emit(cmtat, 'Transfer')
        .withArgs(address1.address, address2.address, 1n)
      return
    }

    // Vulnerable state: address(0) was written into the frozen-address mapping/list,
    // while the global pause switch remains off. This should not be enough to stop
    // ordinary direct ERC20 transfers; if it is, the ENFORCER_ROLE has bypassed PAUSER_ROLE.
    expect(await cmtat.isFrozen(ethers.ZeroAddress)).to.equal(true)
    expect(await cmtat.paused()).to.equal(false)

    // Secure expectation: targeted account freezing must not globally disable all direct
    // transfers. On the vulnerable code this assertion fails because transfer() uses
    // address(0) as the spender sentinel and ValidationModule rejects frozen spenders.
    await expect(cmtat.connect(address1).transfer(address2.address, 1n))
      .to.emit(cmtat, 'Transfer')
      .withArgs(address1.address, address2.address, 1n)
  })
})
```

4.2.6. Incomplete Mint Validation

MEDIUM

Unit Name: `_mintOverride`

Location: contracts/modules/0_CMTATBaseCommon.sol:CMTATBaseCommon._mintOverride

DESCRIPTION

Base _checkTransferred default validates only the source active balance and ignores the mint recipient; a deployment inheriting the common base without the RuleEngine/Allowlist override would mint without enforcing recipient freeze/deactivation/allowlist/rule-engine checks.

User Verdict: True Positive

Verdict Reason: Intercept demo: confirmed TP from prior scan report

EVIDENCE

- Reproduced from the 24 Jun 2026 BugPoCer Scan Report (commit 49544f4), re-seeded for intercept demo.

PROOF OF CONCEPT

PoC reproduces the issue (compiles, fails on vulnerable code).

```
const { expect } = require('chai')
const { ethers } = require('hardhat')

const TERMS = [
  'doc1',
  'https://example.com/doc1',
  '0x6a12eff2f559a5e529ca2c563c53194f6463ed5c61d1ae8f8731137467ab0279'
]
const ZERO = ethers.ZeroAddress

/*
 * Vulnerability: incomplete mint validation via base _checkTransferred (CMTATBaseCommon._mintOverride).
 * Mint validation runs _checkTransferred(address(0), address(0), to, value); over-freezing the
 * address(0) sentinel makes the active-balance check underflow and bricks every mint path.
 */
describe('PoC: freezing address(0) bricks minting through base validation', function () {
  it('mint must still succeed when a benign frozen amount is set on address(0)', async function () {
    const [deployer, admin, forwarder, holder] = await ethers.getSigners()
    const cmtat = await ethers.deployContract('CMTATStandalone', [
      forwarder.address, admin.address, ['CMTA Token', 'CMTAT', 0], ['CMTAT_ISIN', TERMS, 'CMTAT_info'], [ZERO]
    ])
    await cmtat.waitForDeployment()
    await cmtat.connect(admin).setFrozenTokens(ZERO, 1n)
    // FIXED: mint succeeds (sentinel ignored). VULNERABLE: mint reverts -> assertion FAILS.
    await expect(cmtat.connect(admin).mint(holder.address, 10n)).to.not.be.reverted
  })
})
```

4.2.7. Snapshot Hook After State Update

MEDIUM

Unit Name: `_update`

Location: contracts/modules/0_CMTATBaseCommon.sol:_update

DESCRIPTION

ERC20 balances and totalSupply are updated BEFORE the external snapshot hook, so the engine (or a reentrant call) observes post-transfer state, violating pre-update snapshot ordering and corrupting historical records.

User Verdict: True Positive

Verdict Reason: Intercept demo: confirmed TP from prior scan report

EVIDENCE

- Reproduced from the 24 Jun 2026 BugPoCer Scan Report (commit 49544f4), re-seeded for intercept demo.

4.3. Low

4.3.1. External Call Dos

LOW

Unit Name: `_update`

Location: contracts/modules/0_CMTATBaseCommon.sol:_update()

Touchpoints: `SnapshotEngineModule.setSnapshotEngine` (state-source)

DESCRIPTION

When a snapshot engine is configured, every balance-changing operation depends on an unbounded external call to `snapshotEngineLocal.operateOnTransfer`. If the configured snapshot engine reverts, runs out of gas, or is set to a contract that does not correctly implement the hook, the entire transfer/mint/burn operation reverts because the call is not isolated with `try/catch` or a bounded-gas/fallback path. This lets a broken or malicious snapshot-engine dependency halt token transfers and supply operations until a privileged account replaces or disables the engine.

User Verdict: True Positive

Verdict Reason: Intercept demo: matches prior-scan TP

EVIDENCE

- `contracts/modules/0_CMTATBaseCommon.sol:_update()` — the nonzero `snapshotEngineLocal` branch calls `ERC20Upgradeable._update(from, to, amount)` and then unconditionally calls `snapshotEngineLocal.operateOnTransfer(...)`, so hook failure reverts the whole ERC20 state change.
- `contracts/modules/0_CMTATBaseCommon.sol:_update()` — the external hook is reached from the central `_update` override used by transfers, mints, burns, forced transfers, and cross-chain supply changes, making the liveness dependency protocol-wide.
- `contracts/modules/wrapper/extensions/SnapshotEngineModule.sol:setSnapshotEngine()` — the privileged setter only checks that the new engine address differs from the old one and does not validate code presence, interface support, gas behavior, or non-reverting liveness before `_update` begins trusting it.
- `Project invariant: SnapshotEngine balance-update hook MUST be invoked on every balance-changing operation` — the implementation enforces this by reverting on hook failure, which preserves snapshot integrity but turns a snapshot-engine fault into a denial of service for all balance changes.

4.3.2. View Validation Mismatch

LOW

Unit Name: `_canTransferGenericByModule`

Location: contracts/modules/wrapper/controllers/ValidationModule.sol:_canTransferGenericByModule() mint/burn zero-address branches

Touchpoints: `ValidationModuleCore.canTransfer` (entry) → `ValidationModule._canMintBurnByModule` (sink) → `IERC3643ComplianceRead.canTransfer` (state-source)

DESCRIPTION

The public transfer pre-check can report that impossible zero-address transfers are valid because the generic dispatcher treats any `from == address(0)` as a mint and any `to == address(0)` as a burn without first rejecting invalid zero-address transfer inputs. As a result, `canTransfer(nonFrozenUser, address(0), value)` , `canTransfer(address(0), recipient, value)` , or even both-zero inputs can return true when the contract is not deactivated and the checked target is not frozen, even though those are not valid ERC-20 transfers and the actual write path would not execute them as ordinary transfers. Integrators relying on the ERC-3643/7551 view result can therefore accept or queue transactions that are guaranteed to revert or are not authorized transfer operations.

User Verdict: True Positive

Verdict Reason: Intercept demo: matches prior-scan TP

EVIDENCE

- `contracts/modules/wrapper/controllers/ValidationModule.sol:_canTransferGenericByModule()` — the dispatcher classifies inputs solely by `from == address(0)` or `to == address(0)` and has no guard for `to == address(0)` in the mint branch, `from == address(0)` in the burn branch, or both-zero inputs.
- `contracts/modules/wrapper/controllers/ValidationModule.sol:_canMintBurnByModule()` — the mint/burn pre-check only rejects deactivated contracts or frozen targets, so a zero-address counterparty can still produce `true` when not frozen.
- `contracts/modules/wrapper/core/ValidationModuleCore.sol:canTransfer()` — the public ERC-3643 pre-check exposes caller-supplied `from` and `to` values directly to `_canTransferByModule` , making the zero-address misclassification externally observable.
- `contracts/interfaces/tokenization/IERC3643Partial.sol:IERC3643ComplianceRead.canTransfer()` — the interface documents that `canTransfer` returns true only if the transfer is valid, so returning true for zero-address pseudo mint/burn inputs misleads callers about transfer validity.

4.3.3. Incomplete Predicate View

LOW

Unit Name: `ValidationModule`

Location: contracts/modules/wrapper/controllers/ValidationModule.sol:canTransact() and _canTransact() ERC-7943 account predicate

Touchpoints: `ValidationModule.canTransact` (entry) → `ValidationModule._canTransferStandardByModule` (sink) → `ValidationModuleRuleEngine._canTransferWithRuleEngine` (sink) → `PauseModule.deactivated` (state-source)

DESCRIPTION

The ERC-7943 canTransact view overstates account eligibility because it returns true for any non-frozen account while the actual token movement paths enforce additional transaction guards such as pause/deactivation and, in RuleEngine deployments, external RuleEngine checks. During a pause or after deactivation, or when a configured RuleEngine rejects transfers involving the account, canTransact can still report that the account is allowed to transact. Wallets, venues, or compliance systems relying on this account-level predicate can therefore accept or queue transactions for accounts that cannot legally or technically transact under the active token rules. Keep canTransact synchronized with the active validation modules, or document/advertise it only as a narrow frozen/allowlist check rather than a full transaction-eligibility predicate.

User Verdict: True Positive

Verdict Reason: Intercept demo: matches prior-scan TP

EVIDENCE

- `contracts/modules/wrapper/controllers/ValidationModule.sol:canTransact()` — the public ERC-7943 predicate simply delegates to `_canTransact(account)` and is externally callable by any integrator.
- `contracts/modules/wrapper/controllers/ValidationModule.sol:_canTransact()` — the base implementation returns `true` for every account that is not address-frozen and does not read `PauseModule.paused()`, `PauseModule.deactivated()`, or any configured RuleEngine.
- `contracts/modules/wrapper/controllers/ValidationModule.sol:_canTransferStandardByModule()` — the actual standard-transfer predicate rejects transfers when `PauseModule.paused()` is true, so `canTransact` can disagree with transfer enforcement during a pause or deactivation.
- `contracts/modules/wrapper/extensions/ValidationModule/ValidationModuleRuleEngine.sol:_canTransferWithRuleEngine()` — RuleEngine deployments additionally call `ruleEngine.canTransfer(...)`, but `ValidationModule._canTransact()` has no corresponding RuleEngine check.
- `Context: ERC-7943 account eligibility` — `canTransact` is documented as checking whether an account is allowed to transact according to token rules, so omitting active transfer rules makes the view unsafe as an eligibility oracle.

4.3.4. Misleading Access Control Documentation

LOW

Unit Name: `ERC20CrossChainModule`

Location: contracts/modules/wrapper/options/ERC20CrossChainModule.sol:ERC20CrossChainModule.burn(uint256)

Touchpoints: `CMTATBaseERC20CrossChain._authorizeSelfBurn` (sink) → `CMTATBaseERC20CrossChain._authorizeBurnFrom` (sink)

DESCRIPTION

The self-burn entry point documents the wrong access-control modifier. Its NatSpec says the function is protected by `onlyBurnerFrom`, but the implementation uses `onlySelfBurn`, which maps to a different role in the concrete cross-chain base. Operators or bridge integrators following the comment may grant `BURNER_FROM_ROLE` expecting `burn(uint256)` to be callable, while the call actually requires `BURNER_SELF_ROLE` and will revert, potentially breaking self-burn bridge flows.

User Verdict: True Positive

Verdict Reason: Intercept demo: matches prior-scan TP

EVIDENCE

- `contracts/modules/wrapper/options/ERC20CrossChainModule.sol:burn(uint256)` — the NatSpec states `Protected by the modifier onlyBurnerFrom` while the function declaration applies `onlySelfBurn`.
- `contracts/modules/wrapper/options/ERC20CrossChainModule.sol:onlySelfBurn()` — the actual modifier delegates to `_authorizeSelfBurn()` rather than `_authorizeBurnFrom()`.
- `contracts/modules/wrapper/options/ERC20CrossChainModule.sol:onlyBurnerFrom()` — the documented guard is a separate modifier that delegates to `_authorizeBurnFrom()` and is only used by `burnFrom`.
- `contracts/modules/4_CMTATBaseERC20CrossChain.sol:_authorizeSelfBurn()` — the concrete implementation requires `BURNER_SELF_ROLE`, distinct from `_authorizeBurnFrom()` requiring `BURNER_FROM_ROLE`.

4.3.5. Rule Engine Setter Invariant Mismatch

LOW

Unit Name: `ValidationModuleRuleEngineInternal`

Location: contracts/modules/internal/ValidationModuleRuleEngineInternal.sol:_setRuleEngine

DESCRIPTION

The internal RuleEngine setter overwrites the pointer and emits RuleEngine even when unchanged; the documented same-value revert is only in the public wrapper, so the internal sink emits misleading config events.

User Verdict: True Positive

Verdict Reason: Intercept demo: confirmed TP from prior scan report

EVIDENCE

- Reproduced from the 24 Jun 2026 BugPoCer Scan Report (commit 49544f4), re-seeded for intercept demo.

4.3.6. Inconsistent Compliance View

LOW

Unit Name: `_canTransferGenericByModule`

Location: contracts/modules/wrapper/controllers/ValidationModule.sol:_canTransferGenericByModule

DESCRIPTION

Zero-to-zero classified as a mint can return true when active and target not frozen, so canTransfer/canTransferFrom report impossible zero-to-zero movements as compliant.

User Verdict: True Positive

Verdict Reason: Intercept demo: confirmed TP from prior scan report

EVIDENCE

- Reproduced from the 24 Jun 2026 BugPoCer Scan Report (commit 49544f4), re-seeded for intercept demo.

PROOF OF CONCEPT

PoC reproduces the issue (compiles, fails on vulnerable code).

```
const { expect } = require('chai')
const { ethers } = require('hardhat')

const TERMS = [
  'doc1',
  'https://example.com/doc1',
  '0x6a12eff2f559a5e529ca2c563c53194f6463ed5c61d1ae8f8731137467ab0279'
]
const ZERO = ethers.ZeroAddress

/*
 * Vulnerability: inconsistent compliance view (ValidationModule._canTransferGenericByModule).
 * A zero-to-zero request is classified as a mint and can return true even though it is not a
 * valid transfer, so canTransfer reports an impossible movement as compliant.
 */
describe('PoC: canTransfer reports a zero-to-zero movement as valid', function () {
  it('canTransfer(address(0), address(0), 1) must be false', async function () {
    const [deployer, admin, forwarder] = await ethers.getSigners()
    const cmtat = await ethers.deployContract('CMTATStandalone', [
      forwarder.address, admin.address, ['CMTA Token', 'CMTAT', 0], ['CMTAT_ISIN', TERMS, 'CMTAT_info'], [ZERO]
    ])
    await cmtat.waitForDeployment()
    // FIXED: false. VULNERABLE: true -> assertion FAILS.
    expect(await cmtat.canTransfer(ZERO, ZERO, 1n)).to.equal(false)
  })
})
```

4.3.7. Misleading Access Control Documentation Enforcer

LOW

Unit Name: `EnforcementModule`

Location: contracts/modules/wrapper/core/EnforcementModule.sol:onlyEnforcer

DESCRIPTION

onlyEnforcer NatSpec claims it restricts burner functions but gates address-freeze mutations, misleading role reviews about freeze vs burn authority.

User Verdict: True Positive

Verdict Reason: Intercept demo: confirmed TP from prior scan report

EVIDENCE

- Reproduced from the 24 Jun 2026 BugPoCer Scan Report (commit 49544f4), re-seeded for intercept demo.

4.3.8. Zero Address Transfer Validation Mismatch

LOW

Unit Name: `canTransfer`

Location: contracts/modules/wrapper/core/ValidationModuleCore.sol:canTransfer

DESCRIPTION

canTransfer does not reject zero-address endpoints; zero from/to are treated as mint/burn checks so zero-endpoint queries can return true for impossible ERC-20 transfers.

User Verdict: True Positive

Verdict Reason: Intercept demo: confirmed TP from prior scan report

EVIDENCE

- Reproduced from the 24 Jun 2026 BugPoCer Scan Report (commit 49544f4), re-seeded for intercept demo.

PROOF OF CONCEPT

PoC reproduces the issue (compiles, fails on vulnerable code).

```
const { expect } = require('chai')
const { ethers } = require('hardhat')

const TERMS = [
  'doc1',
  'https://example.com/doc1',
  '0x6a12eff2f559a5e529ca2c563c53194f6463ed5c61d1ae8f8731137467ab0279'
]
const ZERO = ethers.ZeroAddress

/*
 * Vulnerability: zero-address transfer validation mismatch (ValidationModuleCore.canTransfer).
 * canTransfer does not reject zero-address endpoints; a zero `from` is treated as a mint and
 * can return true for an impossible ERC-20 transfer.
 */
describe('PoC: canTransfer reports a zero-from transfer as valid', function () {
  it('canTransfer(address(0), holder, 1) must be false', async function () {
    const [deployer, admin, forwarder, holder] = await ethers.getSigners()
    const cmtat = await ethers.deployContract('CMTATStandalone', [
      forwarder.address, admin.address, ['CMTA Token', 'CMTAT', 0], ['CMTAT_ISIN', TERMS, 'CMTAT_info'], [ZERO]
    ])
    await cmtat.waitForDeployment()
    // FIXED: false (invalid endpoint). VULNERABLE: true -> assertion FAILS.
    expect(await cmtat.canTransfer(ZERO, holder.address, 1n)).to.equal(false)
  })
})
```

4.3.9. Misleading Burn Authority Surface

LOW

Unit Name: `burn`

Location: contracts/modules/wrapper/core/ERC20BurnModule.sol:ERC20BurnModule

DESCRIPTION

ERC20BurnModule docs/onlyBurner describe it as all burn functions, but cross-chain adds crosschainBurn/burnFrom/burn under CROSS_CHAIN_ROLE/BURNER_FROM_ROLE/BURNER_SELF_ROLE, so tracking only BURNER_ROLE misses burn authorities.

User Verdict: True Positive

Verdict Reason: Intercept demo: confirmed TP from prior scan report

EVIDENCE

- Reproduced from the 24 Jun 2026 BugPoCer Scan Report (commit 49544f4), re-seeded for intercept demo.

4.3.10. Overfrozen Balance Accounting Corruption

LOW

Unit Name: `ERC20EnforcementModule`

Location: contracts/modules/wrapper/extensions/ERC20EnforcementModule.sol:setFrozenTokens

DESCRIPTION

setFrozenTokens writes an absolute frozen amount without capping to balanceOf or excluding address(0); over-freezing underflow-reverts getActiveBalanceOf/transfer/burn, and freezing address(0) bricks mint paths.

User Verdict: True Positive

Verdict Reason: Intercept demo: confirmed TP from prior scan report

EVIDENCE

- Reproduced from the 24 Jun 2026 BugPoCer Scan Report (commit 49544f4), re-seeded for intercept demo.

PROOF OF CONCEPT

PoC reproduces the issue (compiles, fails on vulnerable code).

```
const { expect } = require('chai')
const { ethers } = require('hardhat')

const TERMS = [
  'doc1',
  'https://example.com/doc1',
  '0x6a12eff2f559a5e529ca2c563c53194f6463ed5c61d1ae8f8731137467ab0279'
]
const ZERO = ethers.ZeroAddress

/*
 * Vulnerability: overfrozen balance accounting corruption (ERC20EnforcementModule.setFrozenTokens).
 * setFrozenTokens writes the ABSOLUTE frozen amount without the `value <= balanceOf(account)`
 * cap that _freezePartialTokens enforces, so getActiveBalanceOf underflows (Panic 0x11).
 */
describe('PoC: setFrozenTokens above balance corrupts active-balance accounting', function () {
  it('a frozen amount greater than the balance must not brick getActiveBalanceOf', async function () {
    const [deployer, admin, forwarder, holder] = await ethers.getSigners()
    const cmtat = await ethers.deployContract('CMTATStandalone', [
      forwarder.address, admin.address, ['CMTA Token', 'CMTAT', 0], ['CMTAT_ISIN', TERMS, 'CMTAT_info'], [ZERO]
    ])
    await cmtat.waitForDeployment()
    await cmtat.connect(admin).mint(holder.address, 50n)
    await cmtat.connect(admin).setFrozenTokens(holder.address, 100n)
    // FIXED: getActiveBalanceOf returns a bounded value. VULNERABLE: 50-100 underflows -> assertion FAILS.
    await expect(cmtat.getActiveBalanceOf(holder.address)).to.not.be.reverted
  })
})
```

4.3.11. Cross Interface Event Inconsistency

LOW

Unit Name: `ExtraInformationModule`

Location: contracts/modules/wrapper/extensions/ExtraInformationModule.sol:setTerms

DESCRIPTION

ERC7551 exposes two terms setters over the same storage but only the ERC7551 overload emits Terms(bytes32,string); the inherited ICMTAT setTerms changes ERC7551-observable state with no event, so listeners miss updates.

User Verdict: True Positive

Verdict Reason: Intercept demo: confirmed TP from prior scan report

EVIDENCE

- Reproduced from the 24 Jun 2026 BugPoCer Scan Report (commit 49544f4), re-seeded for intercept demo.

4.3.12. Approve Missing Pause Protection

LOW

Unit Name: `approve`

Location: contracts/modules/0_CMTATBaseCore.sol:CMTATBaseCore.approve

DESCRIPTION

The Light deployment (CMTATBaseCore) does not gate approve with whenNotPaused; allowances can be created/changed while paused, unlike the full variants.

User Verdict: True Positive

Verdict Reason: Intercept demo: confirmed TP from prior scan report

EVIDENCE

- Reproduced from the 24 Jun 2026 BugPoCer Scan Report (commit 49544f4), re-seeded for intercept demo.

PROOF OF CONCEPT

PoC reproduces the issue (compiles, fails on vulnerable code).

```
const { expect } = require('chai')
const { ethers } = require('hardhat')

const TERMS = [
  'doc1',
  'https://example.com/doc1',
  '0x6a12eff2f559a5e529ca2c563c53194f6463ed5c61d1ae8f8731137467ab0279'
]
const ZERO = ethers.ZeroAddress

/*
 * Vulnerability: approve missing pause protection (CMTATBaseCore / Light approve()).
 * The Light deployment does not gate approve with whenNotPaused, so allowances can be
 * created while the token is paused, unlike the full variants.
 */
describe('PoC: Light approve() is callable while paused', function () {
  it('approve must revert while the contract is paused', async function () {
    const [deployer, admin, holder, spender] = await ethers.getSigners()
    const cmtat = await ethers.deployContract('CMTATStandaloneLight', [admin.address, ['CMTA Token', 'CMTAT', 0]])
    await cmtat.waitForDeployment()
    await cmtat.connect(admin).pause()
    // FIXED: approve reverts while paused. VULNERABLE: it succeeds -> assertion FAILS.
    await expect(cmtat.connect(holder).approve(spender.address, 100n)).to.be.reverted
  })
})
```